

Parallel Programming Beyond the CPU

Tim Mattson



Tim demonstrates his new invention: Kayak snorkeling. Palawan Philippines, 2019

An brief aside about the origins of parallel programming models

The Bad Old days ...

Parallel programming environments in the 90's

ABCPL	CORRELATE	GLU	Mentat	Parafraise2	pC++
ACE	CPS	GUARD	Legion	Paralation	SCHEDULE
ACT++	CRL	HASL.	Meta Chaos	Parallel-C++	SciTL
Active messages	CSP	Haskell	Midway	Parallaxis	POET
Adl	Cthreads	HPC++	Millipede	ParC	SDDA.
Adsmith	CUMULVS	JAVAR.	CparPar	ParLib++	SHMEM
ADDAP	DAGGER	HORUS	Mirage	ParLin	SIMPLE
AFAPI	DAPPLE	HPC	MpC	Parmacs	Sina
ALWAN	Data Parallel C	HPF	MOSIX	Parti	SISAL.
AM	DC++	IMPACT	Modula-P	pC	distributed smalltalk
AMDC	DCE++	ISIS.	Modula-2*	pC++	SMI.
AppLeS	DDD	JAVAR	Multipol	PCN	SONiC
Amoeba	DICE.	JADE	MPI	PCP:	Split-C.
ARTS	DIPC	Java RMI	MPC++	PH	SR
Athapascan-0b	DOLIB	javaPG	Munin	PEACE	Sthreads
Aurora	DOME	JavaSpace	Nano-Threads	PCU	Strand.
Automap	DOSMOS.	JIDL	NESL	PET	SUIF.
bb_threads	DRL	Joyce	NetClasses++	PETSc	Synergy
Blaze	DSM-Threads	Khoros	Nexus	PENNY	Telegrphos
BSP	Ease .	Karma	Nimrod	Phosphorus	SuperPascal
BlockComm	ECO	KOAN/Fortran-S	NOW	POET.	TCGMSG.
C*.	Eiffel	LAM	Objective Linda	Polaris	Threads.h++.
"C* in C	Eilean	Lilac	Occam	POOMA	TreadMarks
C**	Emerald	Linda	Omega	POOL-T	TRAPPER
CarlOS	EPL	JADA	OpenMP	PRESTO	uC++
Cashmere	Excalibur	WWWinda	Orca	P-RIO	UNITY
C4	Express	ISETL-Linda	OOF90	Prospero	UC
CC++	Falcon	ParLin	P++	Proteus	V
Chu	Filaments	Eilean	P3L	QPC++	ViC*
Charlotte	FM	P4-Linda	p4-Linda	PVM	Visifold V-NUS
Charm	FLASH	Glenda	Pablo	PSI	VPE
Charm++	The FORCE	POSYBL	PADE	PSDM	Win32 threads
Cid	Fork	Objective-Linda	PADRE	Quake	WinPar
Cilk	Fortran-M	LiPS	Panda	Quark	WWWinda
CM-Fortran	FX	Locust	Papers	Quick Threads	XENOOPS
Converse	GA	Lparx	AFAPI.	Sage++	XPC
Code	GAMMA	Lucid	Para++	SCANDAL	Zounds
COOL	Glenda	Maisie	Paradigm	SAM	ZPL
		Manifold			

A combination of vendors wanting to lock-down applications and the fact that its "fun" to create new programming environments created chaos for application developers

History of MPI

Workstation vendors wanted into the HPC market

PVM was great but didn't support quality SW engineering

Hardware:

By the early 90's, massively parallel processors (MPPs) and the new trend with clusters convinced even the skeptics that the "killer micros" had won.

MPP Vendors

Needed a common foundation to build a parallel SW industry

User Community

Fed-up recoding as they moved between platforms

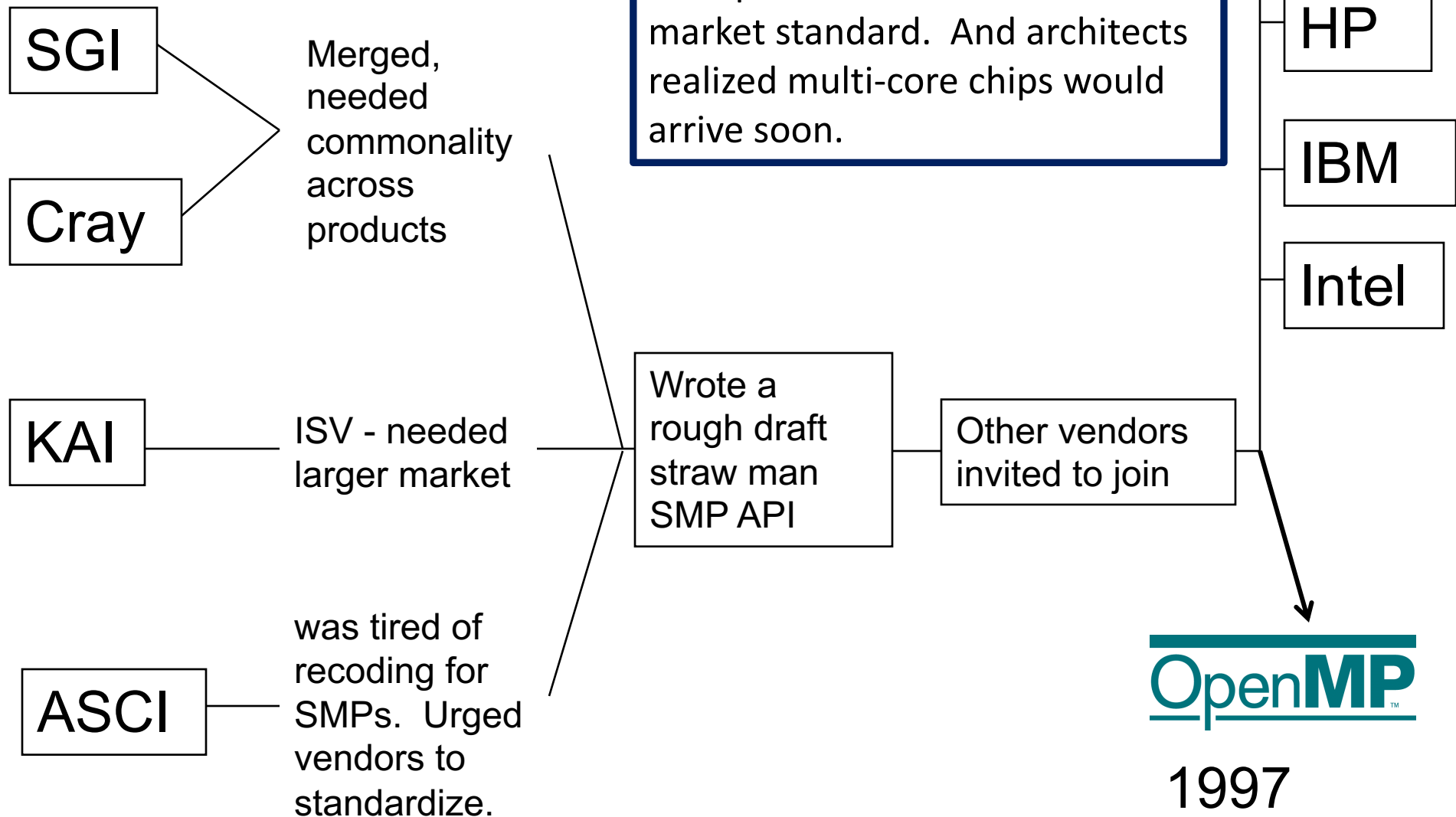
After several years of informal discussions, the MPI forum was created in 1992. A draft specification was presented one year later at SC'93.



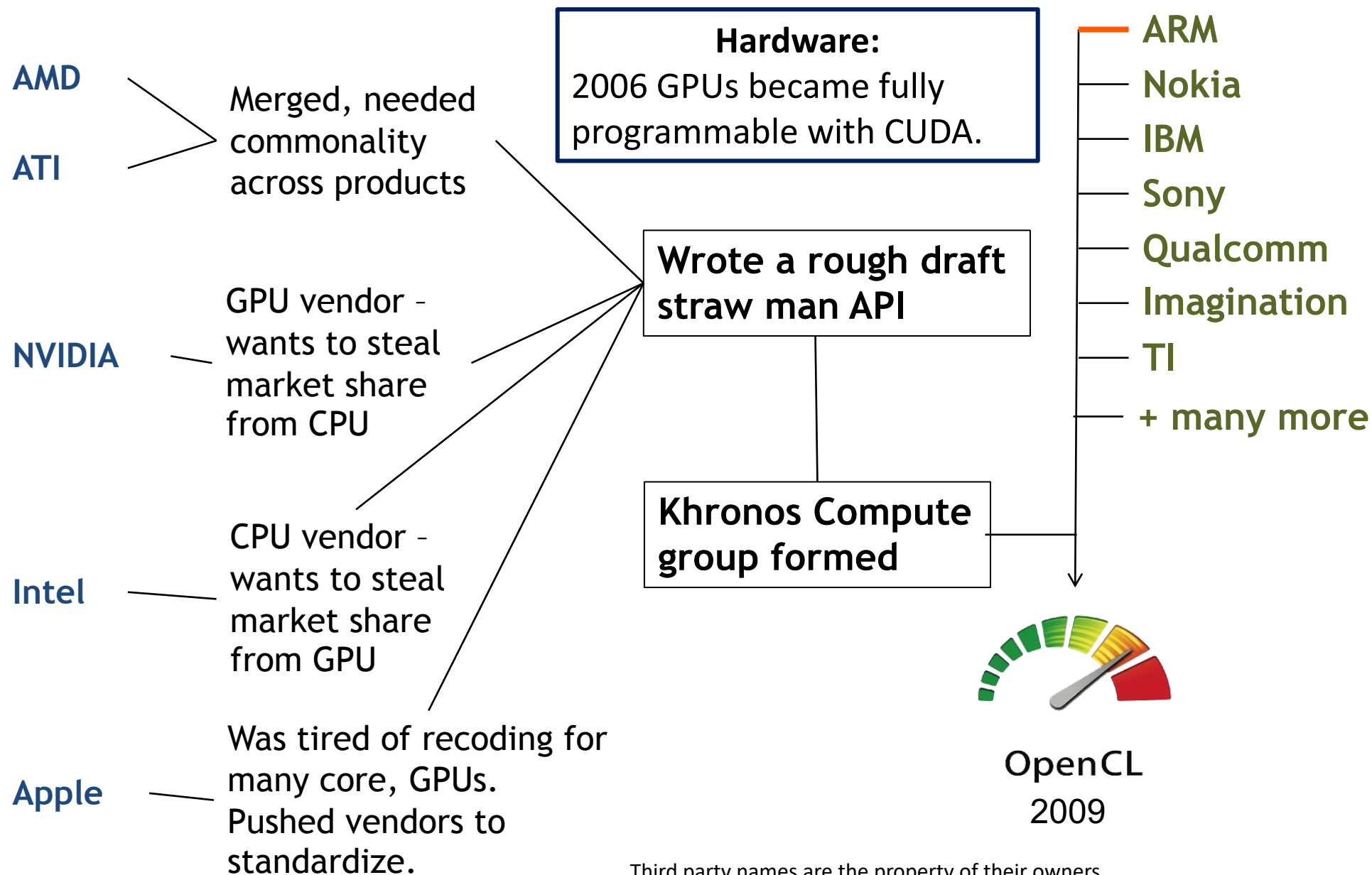
1994

Many of us worked in the MPI forum ... leadership came from the DOE National Labs. In particular, the reference implementation from Bill Gropp and Rusty Lusk of Argonne national lab called MPIch helped us get it right in the 1.0 specification and made sure a working implementation of the standard was available right from the beginning.

History of OpenMP



The origins of OpenCL

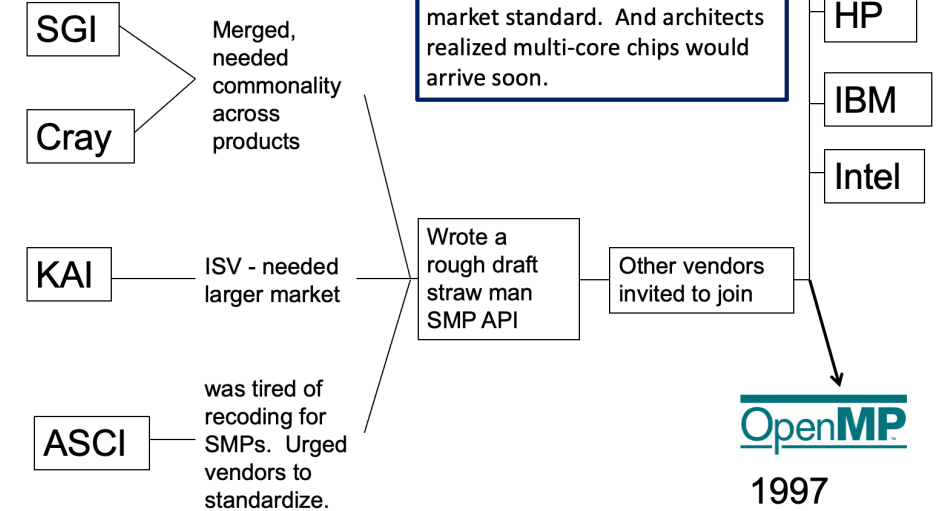


Third party names are the property of their owners.

Key Lesson for you

- Application developers in HPC are the community that really matters You have MUCH MORE power than you think, but only:
 - You are willing to speak with One Voice.
 - You refuse to be trapped in a walled Garden.
 - Vendors will try to divide and conquer you. Hold firm or all is lost!

History of OpenMP



History of MPI

Workstation vendors wanted into the HPC market

PVM was great but didn't support quality SW engineering

Hardware:

By the early 90's, massively parallel processors (MPPs) and the new trend with clusters convinced even the skeptics that the "killer micros" had won.

MPP Vendors

Needed a common foundation to build a parallel SW industry

User Community

Fed-up recoding as they moved between platforms

After several years of informal discussions, the MPI forum was created in 1992. A draft specification was presented one year later at SC'93.



Many of us worked in the MPI forum ... leadership came from the DOE National Labs. In particular, the reference implementation from Bill Gropp and Rusty Lusk of Argonne national lab called MPICH helped us get it right in the 1.0 specification and made sure a working implementation of the standard was available right from the beginning.

The Bad Old days ...

Parallel programming environments in the 90's

ABCPL	CORRELATE	GLU	Mentat	Parafuse2	pC++
ACE	CPS	GUARD	Legion	Paralation	SCHEDULE
ACT++	CRL	HASL	Meta Chaos	Parallel-C++	SciTL
Active messages	CSP	Haskell	Midway	Parallaxis	ScTL
Adl	Cthreads	HPC++	Millipede	ParC	SHMEM
Adsmith	CUMULVS	JAVAR	CparPar	ParLib++	SIMPLE
ADDAP	DAGGER	FORUS	Mirage	ParLin	Sina
AFAP	DAPPLE	HPC	MpC	Parmacs	SISAL
ALWAN	Data Parallel C	HPF	MOSIX	Parti	distributed smalltalk
AM	DC++	IMPACT	Modula-P	pC	SML
AMDC	DCE++	ISIS	Modula-2*	pC++	SONIC
AppLeS	DDD	JAVAR	Multipol	PCN	Split-C
Amoeba	DICE	JADE	MPI	PCP	SR
ARTS	DIPC	Java RMI	MPC++	PH	Threads
Athapascan-0b	DOLIB	javaPG	Munin	PEACE	Strand
Aurora	DOME	JavaSpace	Nano-Threads	PCU	SUIF
Automap	DOSMOS	JIDL	JOYCE	PET	Synergy
bb_threads	DRL	Joyce	Khoros	PETSc	SuperPascal
Blaze	DSM-Threads	Khoros		PENNY	TCGMSG
BSP					Threads.h++
BlockComm					TreadMarks
C*					TRAPPER
"C*" in C					
C**					
CarlOS					
Cashmere					
C4					
CC++					
Chu					
Charlotte					
Charm					
Charm++					
Cid					
Cilk					
CM-Fortran					
Converse					
Code					
COOL					

Please don't let this happen again

**EVERYTIME YOU USE CUDA OR
OPENACC**



**GOD KILLS A
KITTEN**

**... Back to our previously scheduled
program**

Parallel Programming Beyond the CPU

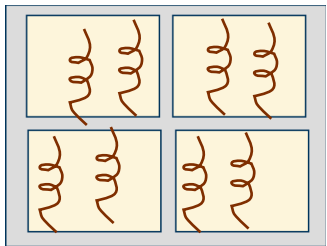
Tim Mattson



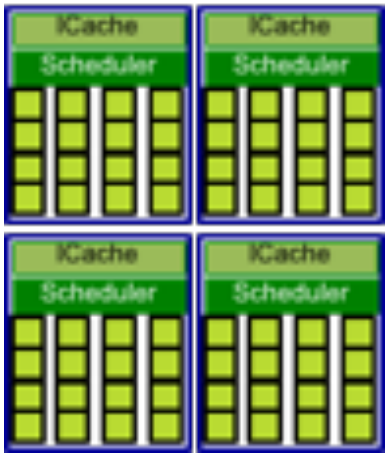
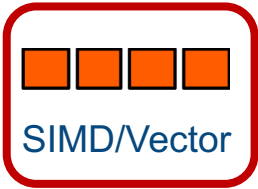
A snorkel

Tim demonstrates his new invention: Kayak snorkeling. Palawan Philippines, 2019

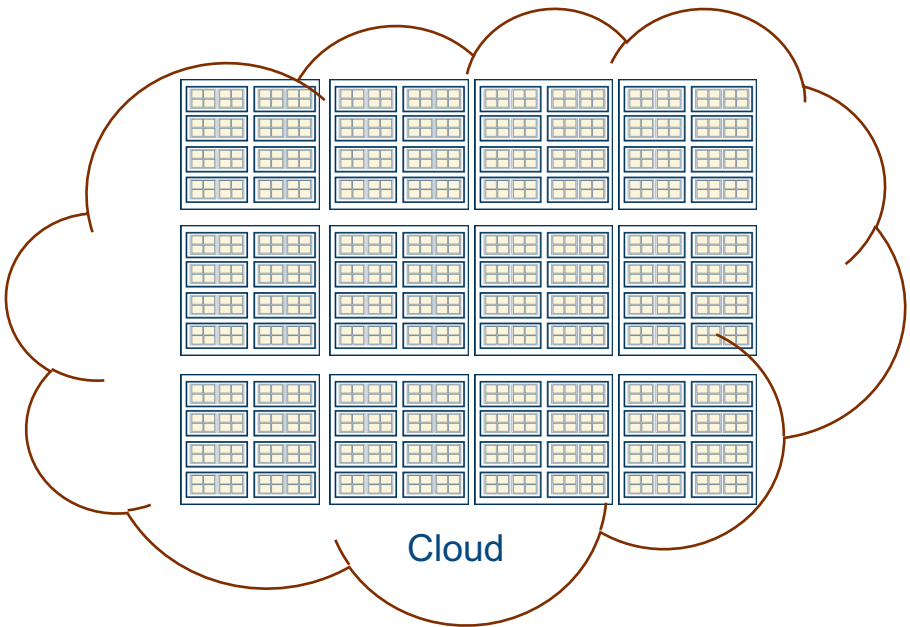
Hardware is diverse



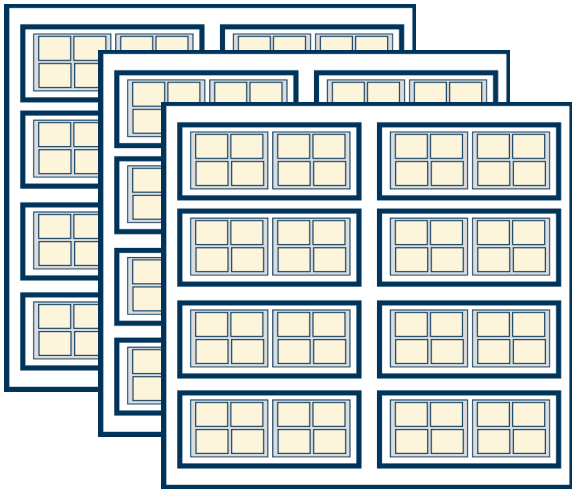
CPU



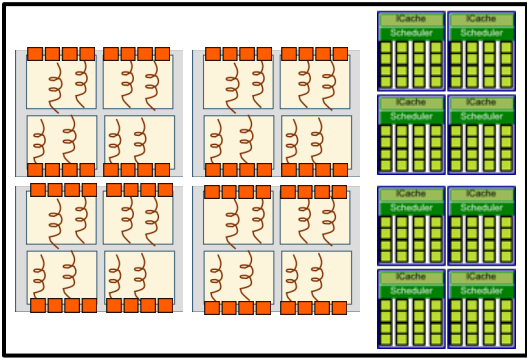
GPU



Cloud



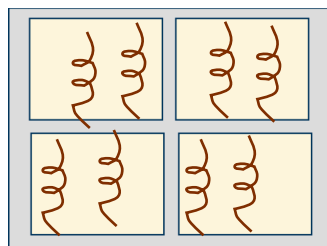
Cluster



Heterogeneous node

Hardware is diverse

You know all about programming with threads and a shared memories



CPU

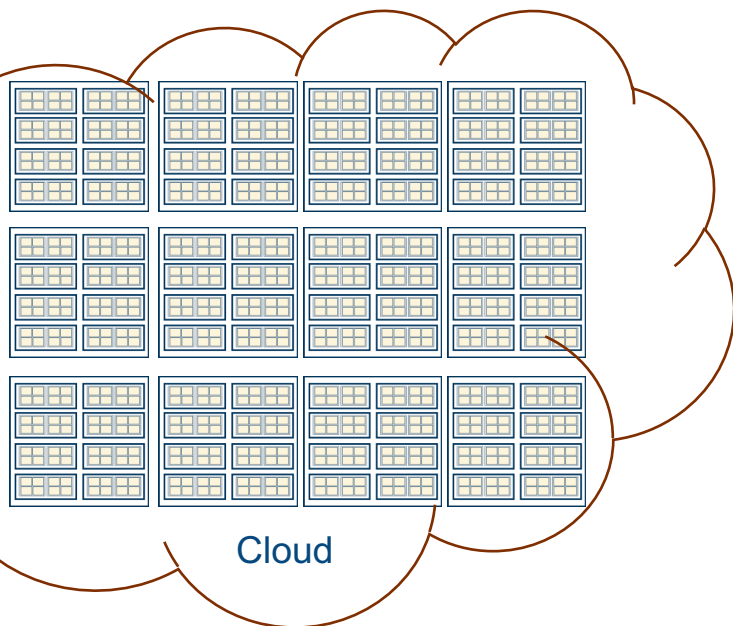
We'll leave vectorization to the compiler (use `-O3`)



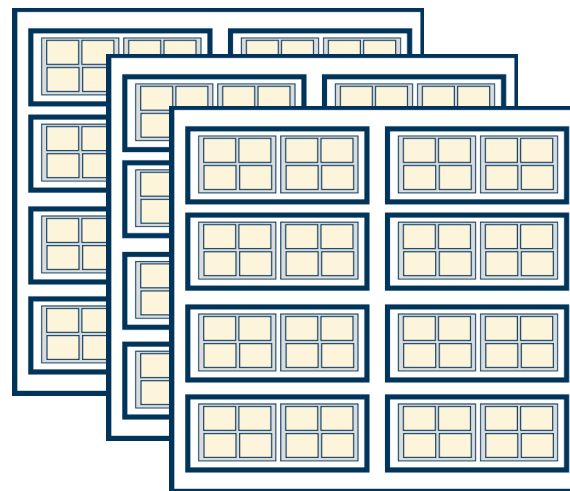
SIMD/Vector



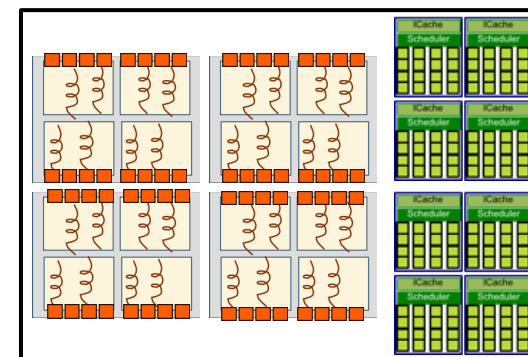
GPU



Cloud

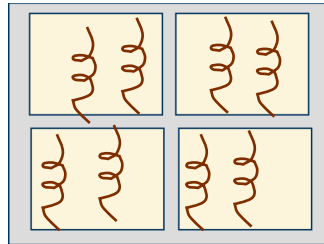


Cluster



Heterogeneous node

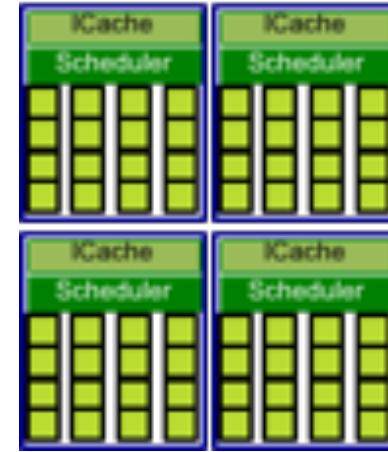
Hardware is diverse



CPU

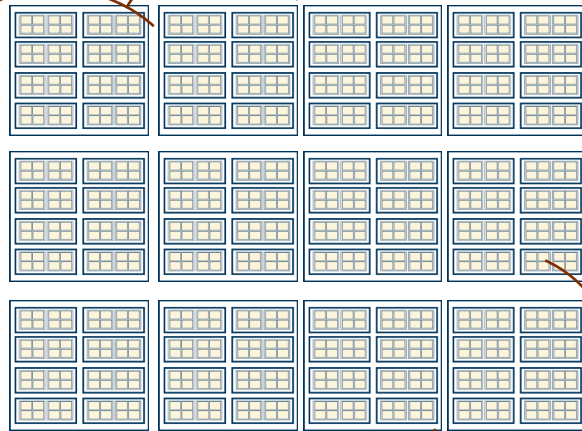


SIMD/Vector

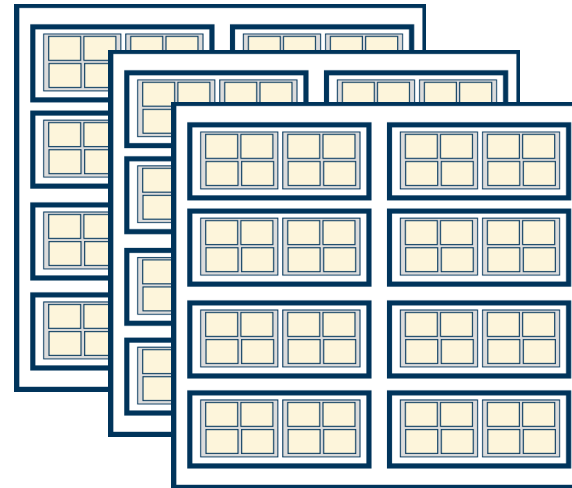


GPU

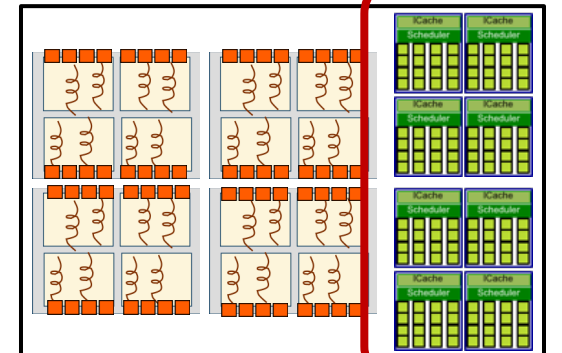
A very brief
introduction to
GPU
programming



Cloud

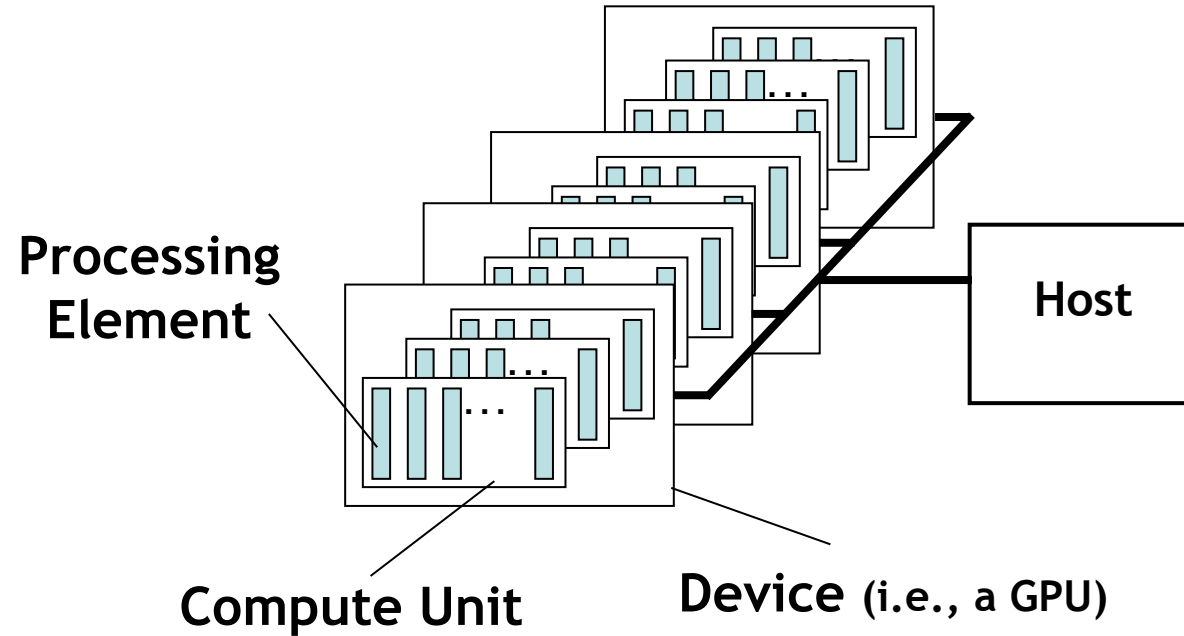


Cluster



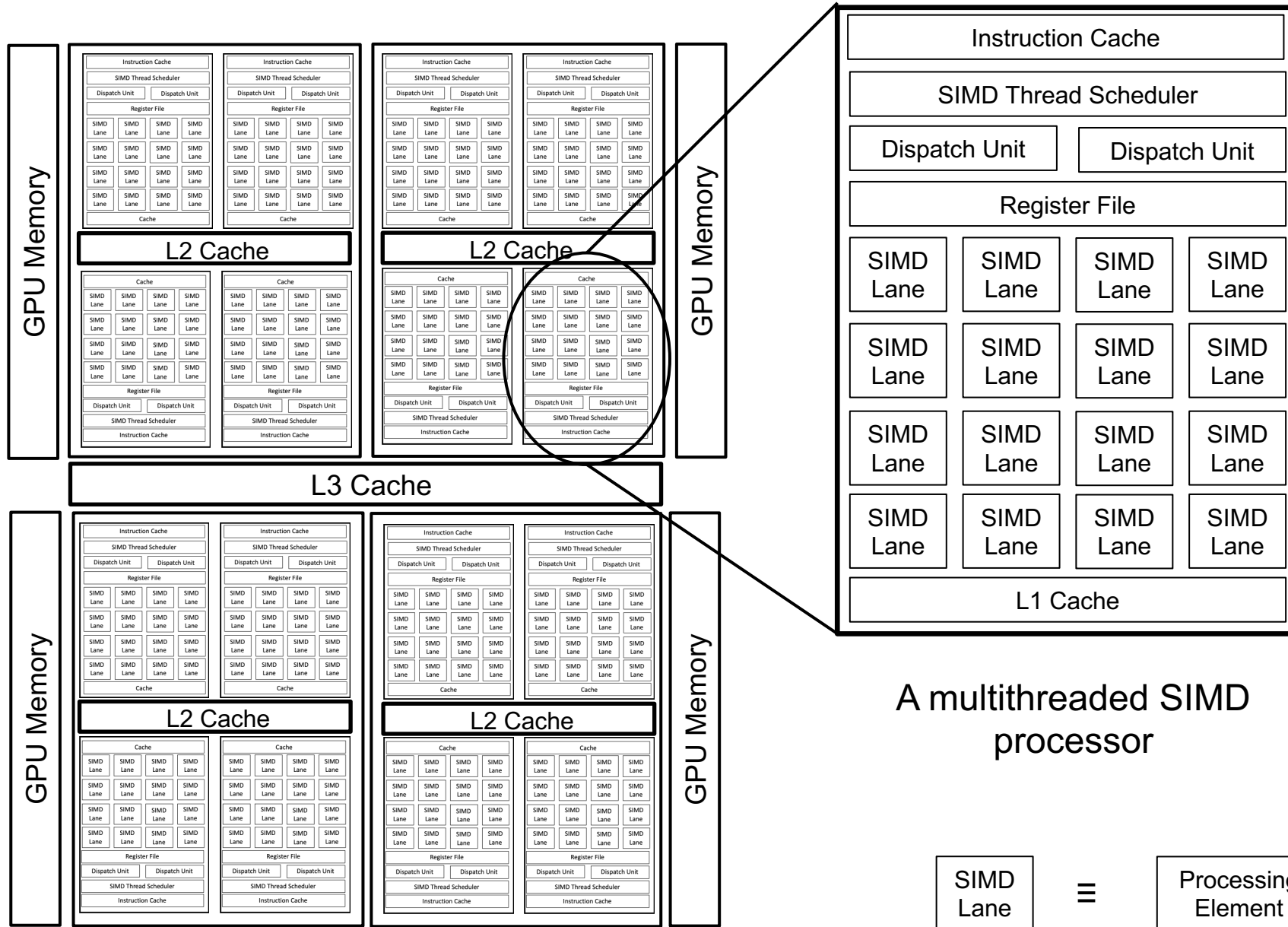
Heterogeneous node

GPU Programming: A Host/Device Platform Model



- One **Host** and one or more **Devices**
 - Each Device is composed of one or more Compute Units
 - Each Compute Unit is divided into one or more **Processing Elements**
- Memory divided into **host memory** and **device memory**

A Generic GPU (following Hennessey and Patterson)



The “BIG idea” Behind GPU programming

Data Parallel vadd with CUDA

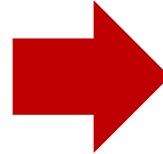
Traditional Loop based vector addition (vadd)

```
int main() {
    int N = . . . ;
    float *a, *b, *c;

    a* =(float *) malloc(N * sizeof(float));

    // ... allocate other arrays (b and c)
    // and fill with data

    for (int i=0;i<N; i++)
        c[i] = a[i] + b[i];
}
```



```
// Compute sum of length-N vectors: C = A + B
void __global__
vecAdd (float* a, float* b, float* c, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) c[i] = a[i] + b[i];
}

int main () {
    int N = ... ;
    float *a, *b, *c;
    cudaMalloc (&a, sizeof(float) * N);
    // ... allocate other arrays (b and c)
    // and fill with data

    // Use thread blocks with 256 threads each
    vecAdd <<< (N+255)/256, 256 >>> (a, b, c, N);
}
```

Assume a GPU with
unified shared memory
... allocate on host,
visible on device too

How do we execute code on a GPU: The SIMT model (Single Instruction Multiple Thread)

1. Write kernel code for the scalar work-items

```
// Compute sum of order-N matrices: C = A + B
void __global__
matAdd (float* a, float* b, float* c, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;
    if (i < N && j < N) c[i][j] = a[i][j] + b[i][j];
}

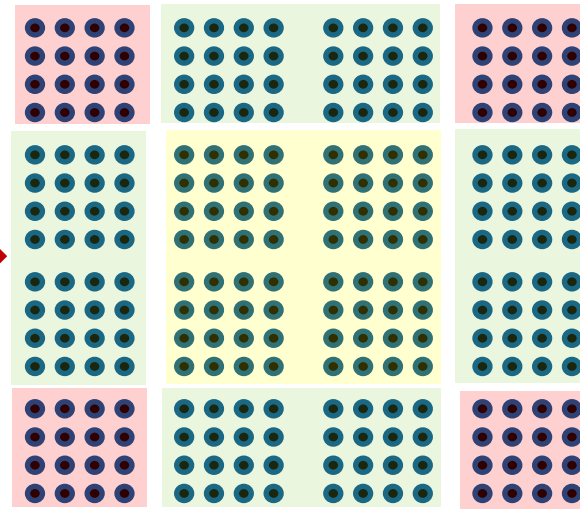
int main () {
    int N = ... ;
    float *a, *b, *c;
    cudaMalloc (&a, sizeof(float) * N);
    // ... allocate other arrays (b and c)
    // and fill with data

    // define threadBlocks and the Grid
    dim3 dimBlock(4,4);
    dim3 dimGrid(4,4);

    // Launch kernel on Grid
    matAdd <<< dimGrid,dimBlock>>> (a, b, c, N);
}
```

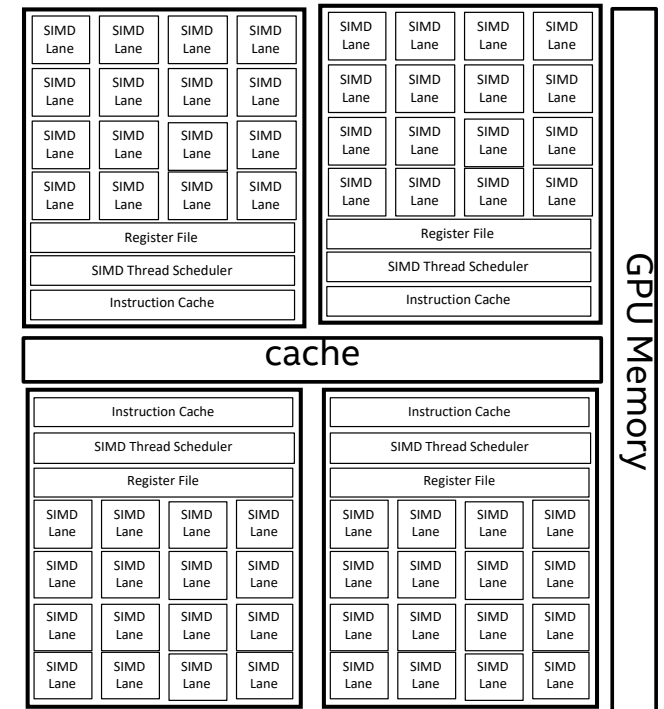
This is CUDA code

2. Map work-items onto an N dim index space.



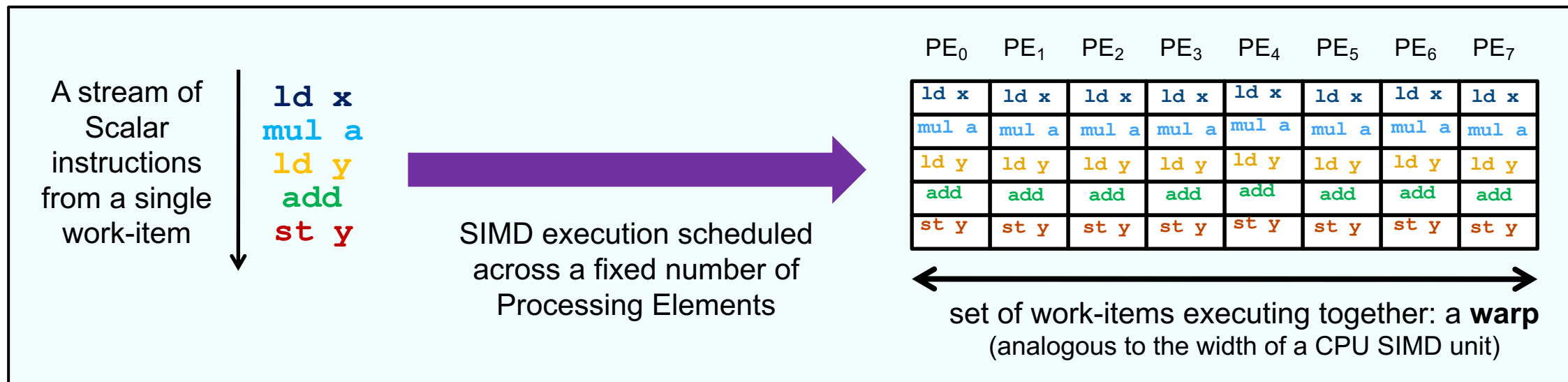
3. Map data structures onto the same index space

4. Run on hardware designed around the same SIMT execution model



SIMT: One instruction stream maps onto many Processing Elements

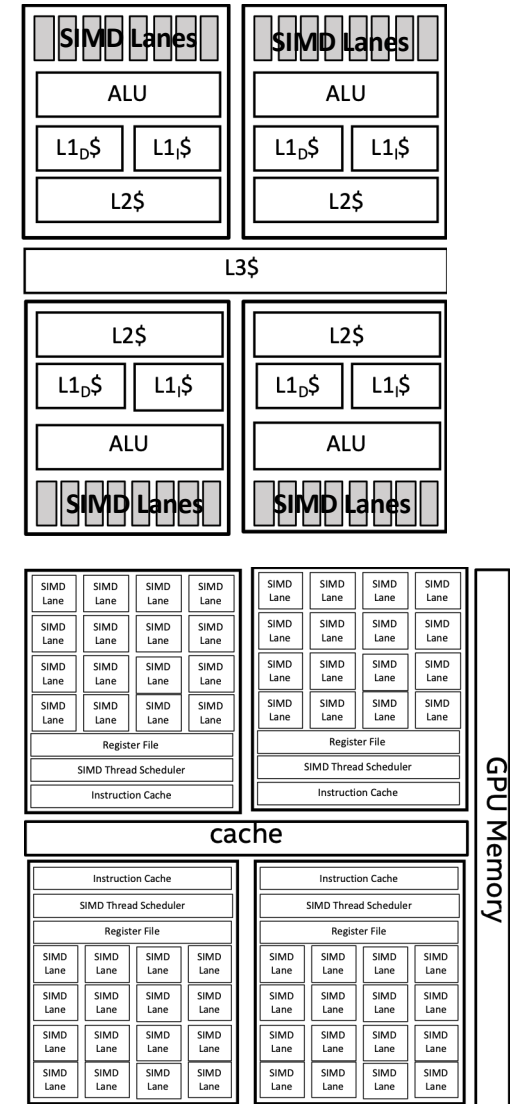
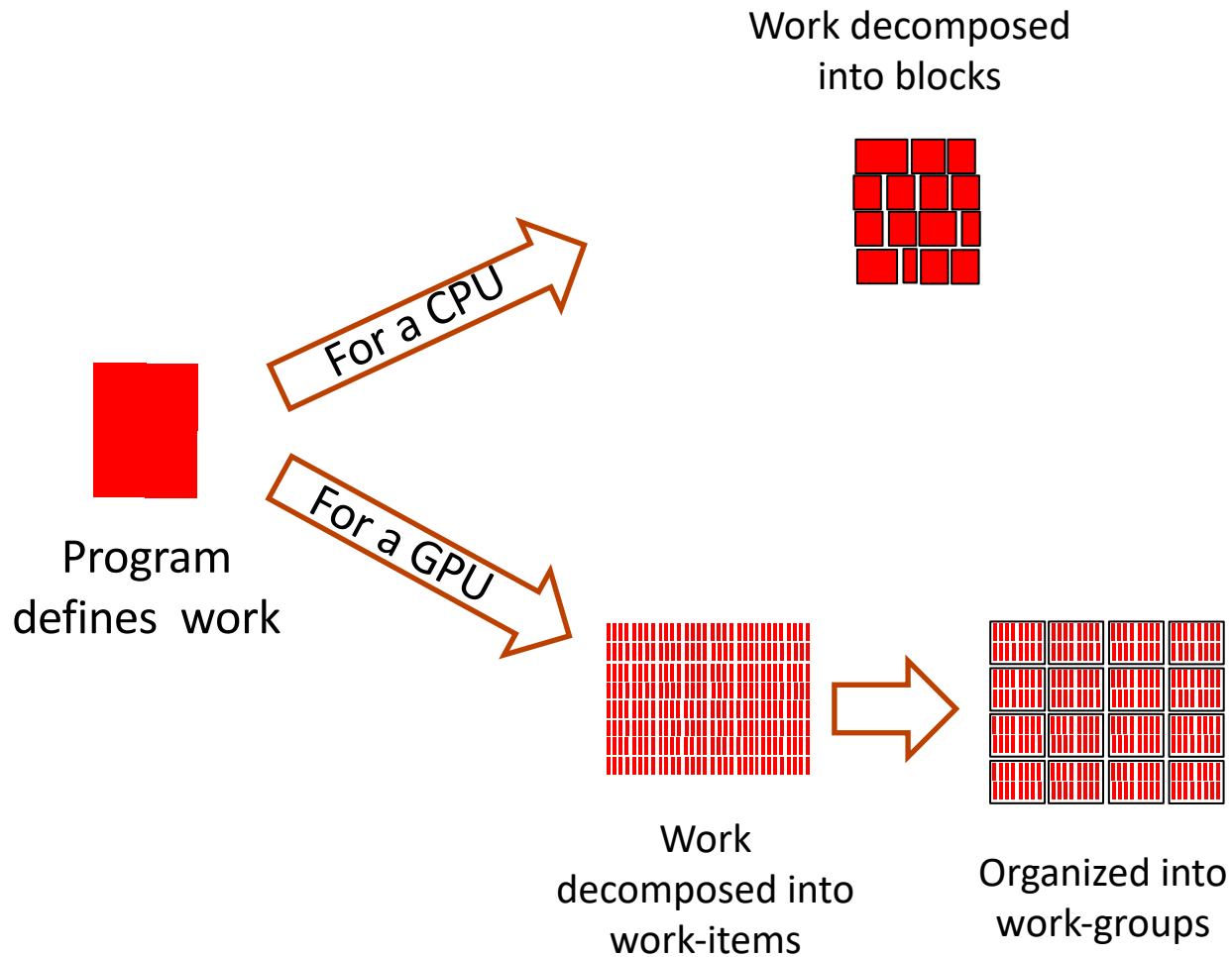
- SIMT model: Individual scalar instruction streams are grouped together for SIMD execution on hardware



GPU nomenclature is really messed up. (sorry about that ... we tried to unify around OpenCL but failed).

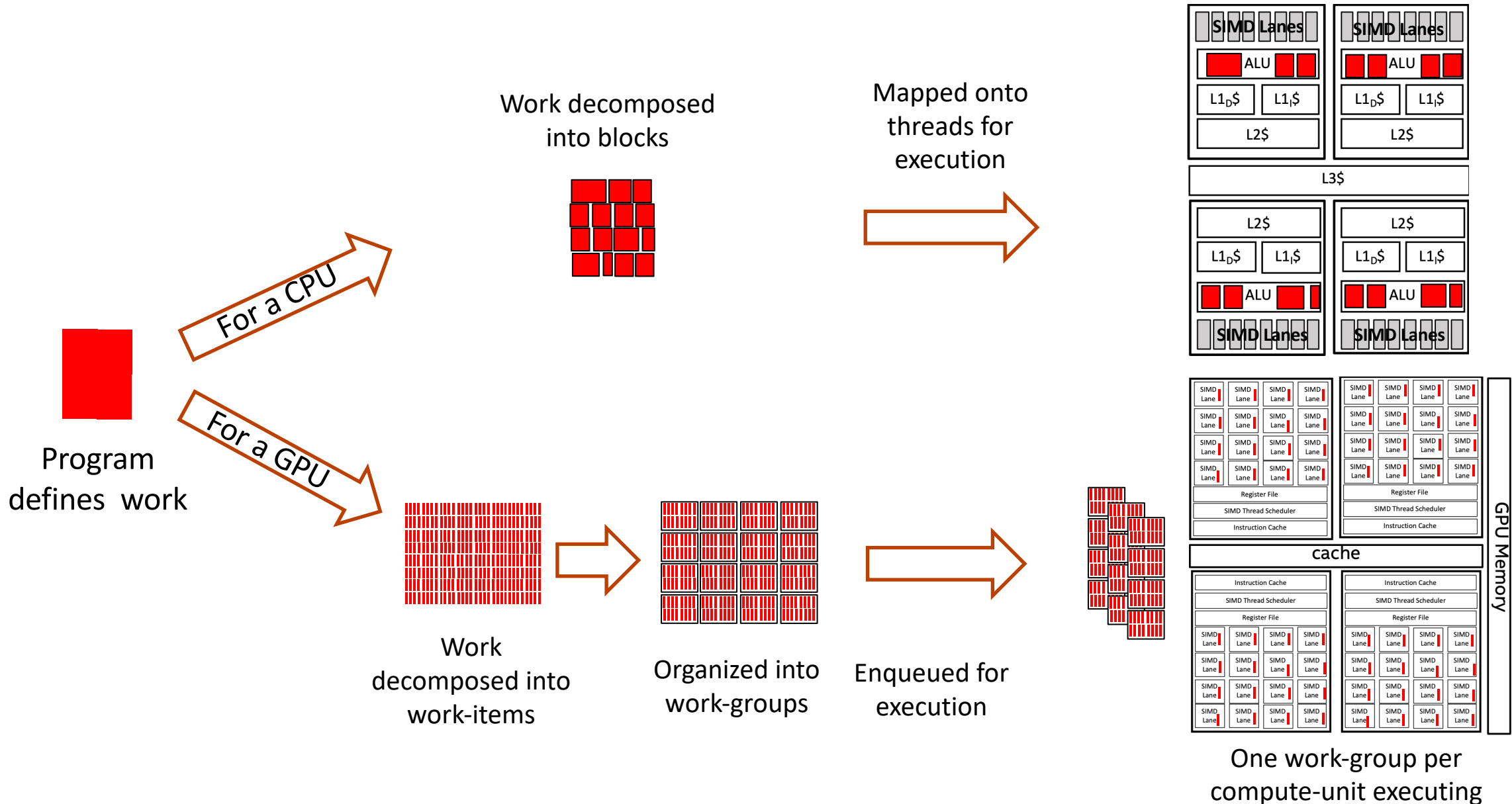
Instruction stream at finest grain	Work-item, CUDA Thread	These names are particularly awful since they conflict with established names from CPU Computing.
Blocks for scheduling work-items	work-group, thread block	
Execution width for work-items	Subgroup, warp	
Finest grained processing element (PE) in a GPU	SIMD Lane, Processing Element, CUDA Core	
Block of PEs driven by a single Instruction sequencer	multithreaded SIMD processor, compute unit, Streaming multiprocessor	

Executing a program on CPUs and GPUs



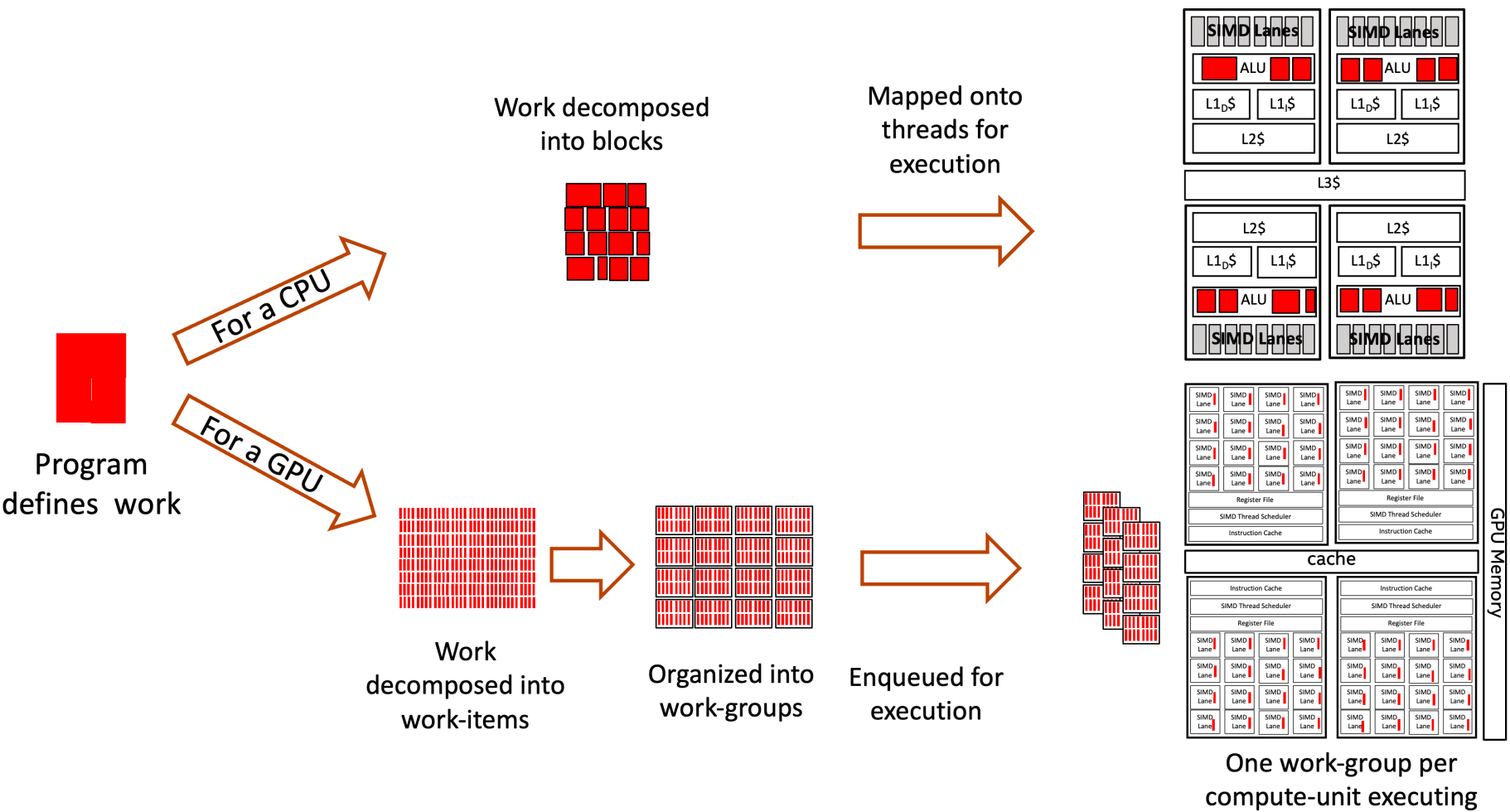
One work-group per compute-unit executing

Executing a program on CPUs and GPUs



CPU/GPU execution models

Executing a program on CPUs and GPUs



For a CPU, the threads are all active and able to make forward progress. Optimized for latency sensitive problems

For a GPU, any given work-group might be in the queue waiting to execute. Optimized for high throughput workflows

5-point stencil: the heat program

- The heat equation models changes in temperature over time.

$$\frac{\partial u}{\partial t} - \alpha \nabla^2 u = 0$$

- We'll solve this numerically on a computer using an explicit **finite difference** discretisation.
- $u = u(t, x, y)$ is a function of space and time.
- Partial differentials are approximated using diamond difference formulae:

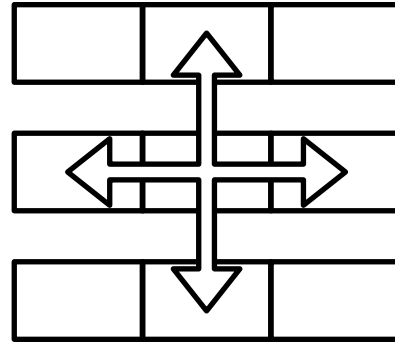
$$\frac{\partial u}{\partial t} \approx \frac{u(t + 1, x, y) - u(t, x, y)}{dt}$$

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u(t, x + 1, y) - 2u(t, x, y) + u(t, x - 1, y)}{dx^2}$$

- Forward finite difference in time, central finite difference in space.

5-point stencil: the heat program

- Given an initial value of u , and any boundary conditions, we can calculate the value of u at time $t+1$ given the value at time t .
- Each update requires values from the north, south, east and west neighbours only:



- Computation is essentially a weighted average of each cell and its neighbouring cells.
- If on a boundary, look up a boundary condition instead.

Heat program ...

```
// Loop over time steps
for (int t = 0; t < nsteps; ++t) {

    // solve over spatial domain for step t
    solve(n, alpha, dx, dt, u, u_tmp);

    // Pointer swap to get ready for next step
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
}
```

Note: Swapping pointers on the host before entering the target region on the next iteration works on a GPU.

When you map pointers between the host and the device, OpenMP remembers the address.

Swapped addresses on the hosts swaps addresses on the device

- Takes two optional command line arguments: <ncells> <nsteps>
 - E.g. ./heat 1000 10
 - 1000x1000 cells, 10 timesteps (the default problem size).
- If no command line arguments are provided, it uses a default:
 - These two commands both run the default problem size of 1000x1000 cells, 10 timesteps.
 - ./heat
 - ./heat 1000 10
- A sensible bigger problem is 8000 x 8000 cells and 10 timesteps.

Heat diffusion problem: 5-point stencil code

```
const double r = alpha * dt / (dx * dx);  
const double r2 = 1.0 - 4.0*r;
```

```
// malloc and initialize u_tmp and u (code not shown)
```

```
for (int t = 0; t < nsteps; ++t) {
```

Loop over time steps

```
    for (int i = 0; i < n; ++i) {
```

```
        for (int j = 0; j < n; ++j) {
```

Loop over NxN spatial domain

```
            u_tmp[i+j*n] = r2 * u[i+j*n] +  
                r * ((i < n-1) ? u[i+1+j*n] : 0.0) +  
                r * ((i > 0) ? u[i-1+j*n] : 0.0) +  
                r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +  
                r * ((j > 0) ? u[i+(j-1)*n] : 0.0);
```

Update the 5-point stencil. Boundary conditions on the edges of the domain are fixed at zero.

```
        }
```

```
    }
```

```
    // Pointer swap to get ready for next step
```

```
    tmp = u;
```

```
    u = u_tmp;
```

```
    u_tmp = tmp;
```

```
}
```

```
}
```

Heat diffusion problem: 5-point stencil code

```
const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;
```

```
// malloc and initialize u_tmp and u (code not shown)
```

```
for (int t = 0; t < nsteps; ++t) {
```

```
    #pragma omp parallel for collapse(2)
```

```
    for (int i = 0; i < n; ++i) {
```

```
        for (int j = 0; j < n; ++j) {
```

```
            u_tmp[i+j*n] = r2 * u[i+j*n] +
                r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
                r * ((i > 0) ? u[i-1+j*n] : 0.0) +
                r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
                r * ((j > 0) ? u[i+(j-1)*n] : 0.0);
```

```
        }
```

```
    }
```

```
    // Pointer swap to get ready for next step
```

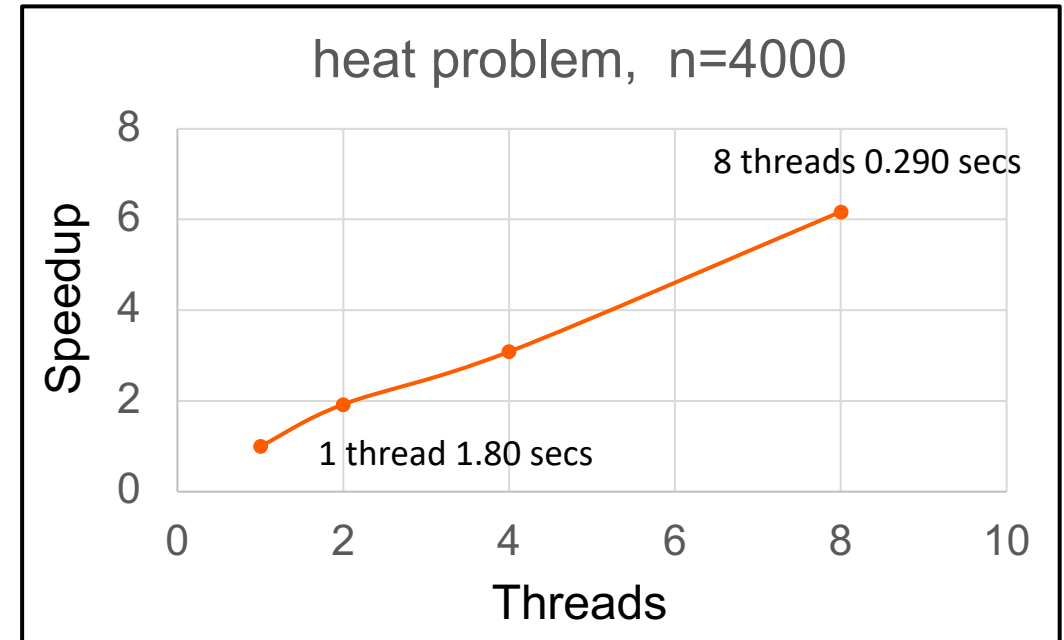
```
    tmp = u;
```

```
    u = u_tmp;
```

```
    u_tmp = tmp;
```

```
}
```

```
}
```



Intel® Xeon™ Gold 5218 @ 2.3 Ghz, 8 cores. Nvidia HPC Toolkit compiler
nvc -fast -fopenmp heat.c

Exercise: parallel stencil (heat)

- Take the provided heat stencil code (heat.c)
- Add OpenMP directives to parallelize the loops on the GPU
- Most of the runtime occurs in the solve() routine. Focus on that function. The rest of the code is there to just support the work inside solve.

- double omp_get_wtime();
- #pragma omp parallel
- #pragma omp for
- #pragma omp parallel for
- #pragma omp task
- #pragma omp taskwait
- #pragma single
- #pragma omp target
- #pragma omp loop
- Plus the clauses

- private(), firstprivate(), reduction(+:var), collapse(n)
- map(to:vp[Lower:Count]) map(from:vp[Lower:Count]) map(tofrom:vp[Lower:Count])

Fetch an interactive
session for a GPU node
salloc -G 1

```
module load nvhpc/24.5
nvc -O3 -mp=gpu heat.c -o heat
sbatch job_gpu.sh
```

```
#!/bin/bash
#SBATCH -p mit_normal_gpu
#SBATCH -t 00:10:00
#SBATCH -n 1
#SBATCH -G 1

module load nvhpc/24.5
OMP_TARGET_OFFLOAD=MANDATORY ./heat
```

job_gpu.sh

Default is tofrom: map(vp[Lower:Count])

If the job is pending too long, try to switch the partition to -p mit_preemptable , which may start faster.

Heat diffusion problem: 5-point stencil code

```

const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;

// malloc and initialize u_tmp and u (code not shown)

for (int t = 0; t < nsteps; ++t) {
    #pragma omp target map(tofrom: u[0:n*n], u_tmp[0:n*n])
    #pragma omp loop
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            u_tmp[i+j*n] = r2 * u[i+j*n]
                           +
                           r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
                           r * ((i > 0) ? u[i-1+j*n] : 0.0) +
                           r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
                           r * ((j > 0) ? u[i+(j-1)*n] : 0.0);
        }
    }
    // Pointer swap to get ready for next step
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
}
}

```

When you map pointers between the host and the device, OpenMP remembers the address.

Swapped addresses on the hosts swaps
addresses on the device

GPU Solver time = 1.40 secs

This isn't much better than the
runtime for a single CPU (1.8 secs)
and worse than 8 cores on a CPU
(0.29 secs).

Why is the performance so bad?

NVIDIA T4 GPU, 16 Gbyte, Turing Arch.
Nvidia HPC Toolkit compiler
nvc -fast -mp=gpu -gpu=cc75 heat.c

Heat diffusion problem: 5-point stencil code

```
const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;

// malloc and initialize u_tmp and u (code not shown)
```

```
for (int t = 0; t < nsteps; ++t) {
    #pragma omp target map(tofrom: u[0:n*n], u_tmp[0:n*n])
    #pragma omp loop
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            u_tmp[i+j*n] = r2 * u[i+j*n]
                           +
                           r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
                           r * ((i > 0) ? u[i-1+j*n] : 0.0) +
                           r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
                           r * ((j > 0) ? u[i+(j-1)*n] : 0.0);
        }
    }
    // Pointer swap to get ready for next step
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
}
}
```

With a runtime of 1.4 secs (worse than the CPU time) we see that Data Movement dominates performance.

At the beginning of each iteration, **copy** $(2*N^2)*\text{sizeof}(\text{TYPE})$ bytes **to** the device

We need to create a **data region** on the GPU that is distinct from the target region.

That way, we can keep the data on the device between target constructs

At the end of each iteration, **copy** $(2*N^2)*\text{sizeof}(\text{TYPE})$ bytes **from** the device

Finer control over data movement

- Recall that data is mapped to/from the device at start/end of target region
 - `#pragma omp target map(tofrom: A[0:N])`
 {
 ...
 }
- Inefficient to move data around all the time
- We need to keep data resident on the device *between* target regions.
- We need to manage the device data environment independently from the target regions that launch kernels.

Target enter/exit data constructs

- Create a data region on the target device (a device data environment) with two standalone directives:

```
#pragma omp target enter data map(...)  
#pragma omp target exit data map(...)
```

- The **target enter data** maps variables to the device data environment.
- The **target exit data** unmaps variables from the device data environment.
- Once created, subsequent **target** regions inherit the existing data environment.

Target enter/exit data example

```
void init_array(int *A, int *B, int N) {  
    for (int i = 0; i < N; ++i) { A[i] = i; B[i]=2*i;}
```

```
    #pragma omp target enter data map(to: A[0:N], B[0:N])  
}
```

```
int main(void) {  
  
    int N = 1024;  
    int *A = malloc(sizeof(int) * N);  
    int *B = malloc(sizeof(int) * N);  
    init_array(A, B, N);
```

```
    #pragma omp target
```

```
    #pragma omp loop
```

```
    for (int i = 0; i < N; ++i)  
        A[i] = A[i] * B[i];
```

```
    #pragma omp target exit data map(from: A[0:N])  
}
```

Exercise

- Modify your parallel heat code from the last exercise.
- Use the 'target data' constructs to control the device data environment.
- Think about the map clauses and how to use them to minimize data movement.
- Question ... will the pointer swap on the host still work?
 - #pragma omp target
 - #pragma omp target enter data map(to: list)
 - #pragma omp target exit data map(from: list)
 - #map(to:list) map(from:list) map(tofrom:list)
 - #pragma omp target teams loop

If your runtimes are tiny, you may want to work with a larger problem.

Try a 4000 by 4000 grid for 100 steps:

```
./heat 4000 100
```

```
module load nvhpc/24.5  
nvc -O3 -mp=gpu heat.c -o heat  
sbatch job_gpu.sh
```

```
#!/bin/bash  
#SBATCH -p mit_normal_gpu  
#SBATCH -t 00:10:00  
#SBATCH -n 1  
#SBATCH -G 1  
  
module load nvhpc/24.5  
OMP_TARGET_OFFLOAD=MANDATORY ./heat
```

Heat diffusion problem: 5-point stencil code

```

const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;
// malloc and initialize u_tmp and u (code not shown)
#pragma omp target enter data map(to: u[0:n*n], u_tmp[0:n*n])

for (int t = 0; t < nsteps; ++t) {
    #pragma omp target
    #pragma omp loop
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            u_tmp[i+j*n] = r2 * u[i+j*n] +
                r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
                r * ((i > 0) ? u[i-1+j*n] : 0.0) +
                r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
                r * ((j > 0) ? u[i+(j-1)*n] : 0.0);
        }
    }
    // Pointer swap to get ready for next step
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
}
#pragma omp target exit data map(from: u[0:n*n])
}

```

Create a data region and map indicated data on entry

GPU Solver time* = **0.42** secs

This is a general principal ...
if you want performance, you
must optimize data
movement.

*includes time for target enter/exit data

Exit the data
region and map
indicated data

NVIDIA T4 GPU, 16 Gbyte, Turing Arch.
Nvidia HPC Toolkit compiler
nvc -fast -mp=gpu -gpu=cc75 heat.c

SIMT Programming models

- CUDA:
 - Released ~2006. Made GPGPU programming “mainstream” and continues to drive innovation in SIMT programming.
 - Downside: proprietary to NVIDIA
- OpenCL:
 - Open Standard for SIMT programming created by Apple, Intel, NVIDIA, AMD, and others. 1st release in 2009.
 - Supports CPUs, GPUs, FPGAs, and DSP chips. The leading cross platform SIMT model.
 - Downside: extreme portability means verbose API. Painfully low level especially for the host-program.
- Sycl:
 - C++ abstraction layer implements SIMT model with kernels as lambdas. Closely aligned with OpenCL. 1st release 2014
 - Downside: Cross platform implementations only emerging recently.
- Directive driven programming models:
 - **OpenACC**: they split from an OpenMP working group to create a competing directive driven API emphasizing descriptive (rather than prescriptive) semantics.
 - Downside: NOT an Open Standard. Controlled by NVIDIA.
 - **OpenMP**: Mixes multithreading and SIMT. Semantics are prescriptive which makes it more verbose. A truly Open standard supported by all the key GPU players.
 - Downside: Poor compiler support so far ... but that will change over the next couple years.

Vector addition with CUDA

```
// Compute sum of length-N vectors: C = A + B
void __global__
vecAdd (float* a, float* b, float* c, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) c[i] = a[i] + b[i];
}

int main () {
    int N = ... ;
    float *a, *b, *c;
    cudaMalloc (&a, sizeof(float) * N);
    // ... allocate other arrays (b and c), fill with data

    // Use thread blocks with 256 threads each
    vecAdd <<< (N+255)/256, 256 >>> (a, b, c, N);
}
```

CUDA kernel as
function

Unified shared
memory ... allocate
on host, visible on
device too

Enqueue the kernel
to execute on the
Grid

Note: Not all GPUs
support a unified shared
memory. We assume its
available here to we don't
need to clutter the code
with explicit data
movement.

Vector addition with SYCL

```
// Compute sum of length-N vectors: C = A + B
```

```
#include <CL/sycl.hpp>
```

```
int main () {
```

```
    int N = ... ;
```

```
    float *a, *b, *c;
```

```
    sycl::queue q;
```

```
    *a = (float *)sycl::malloc_shared(N * sizeof(float), q);
```

```
    // ... allocate other arrays (b and c), fill with data
```

```
    q.parallel_for(sycl::range<1>{N},
```

```
        [=](sycl::id<1> i) {
```

```
            c[i] = a[i] + b[i];
```

```
        });
```

```
    q.wait();
```

```
}
```

Create a queue
for SYCL
commands

Unified shared
memory ... allocate
on host, visible on
device too

Note: Not all
GPUs support a
unified shared
memory. We
assume its
available here so
we don't need to
clutter the code
with explicit data
movement.

Kernel as a C++
Lambda function

[=] means capture external
variables by value.

Vector addition with OpenMP

- Let's add two vectors together ... $C = A + B$

Offload work
onto the GPU
and manage
data movement

```
void vadd(int n, float *a, float *b, float *c)
{
    #pragma omp target map(to:a(0:n),b(0:n)) map(tofrom:c(0:n))
    #pragma omp loop
    for (int i=0; i<n; i++)
        c[i] = a[i] + b[i];
}

int main() {
    float *a, *b, *c; int n = 10000;
    // allocate and fill a and b

    vadd(n, a, b, c);
}
```

Turn the loop
body into a
kernel and run it
on the GPU

Vector addition with OpenACC

- Let's add two vectors together $C = A + B$

Host waits here until the kernel is done. Then the output array `c` is copied back to the host.

```
void vadd(int n,  
          const float *a,  
          const float *b,  
          float *restrict c)  
{  
    int i;  
    #pragma acc parallel loop  
    for (i=0; i<n; i++)  
        c[i] = a[i] + b[i];  
}  
  
int main(){  
    float *a, *b, *c;  int n = 10000;  
    // allocate and fill a and b  
  
    vadd(n, a, b, c);  
}
```

Assure the compiler that `c` is not aliased with other pointers

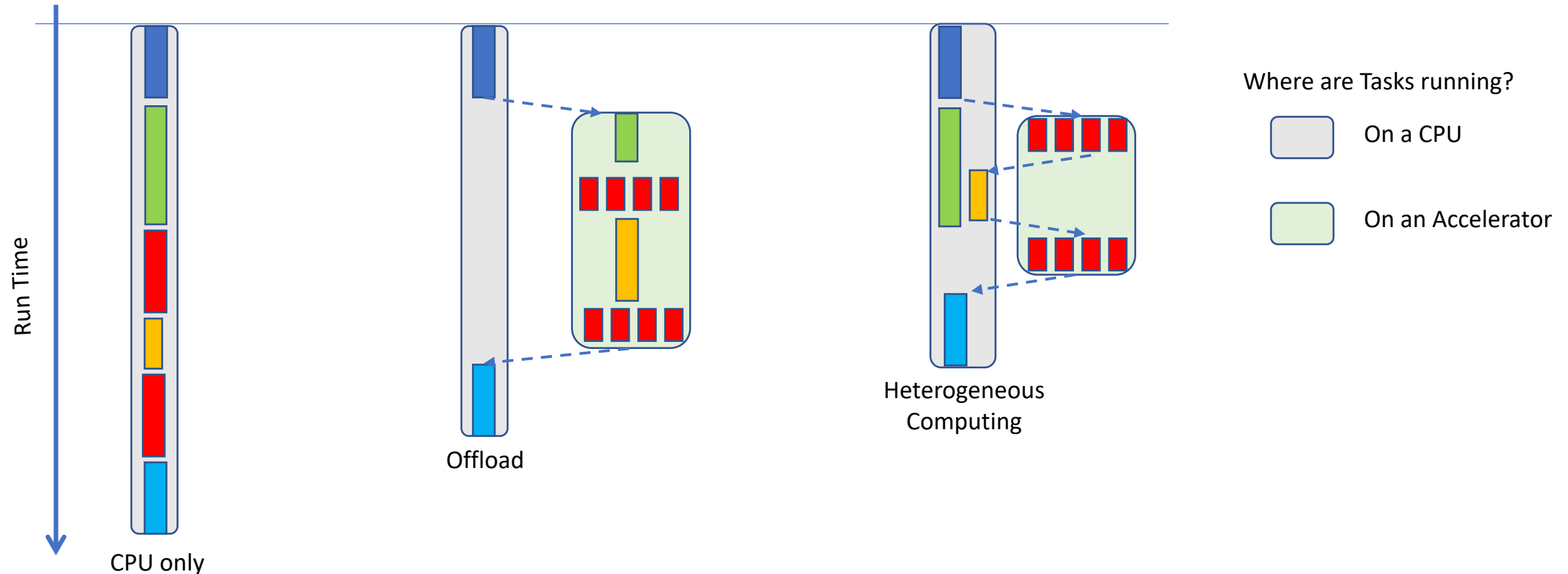
Turn the loop into a kernel, move data to a device, and launch the kernel.

Why so many ways to do the same thing?

- The parallel programming model people have failed you ...
 - It's more fun to create something new in your own closed-community that work across vendors to create a portable API
- The hardware vendors have failed you ...
 - Don't you love my "walled garden"? It's so nice here, programmers, just don't even think of going to some other platform since your code is not portable.
- The standards community has failed you ...
 - Standards are great, but they move too slow. OpenACC stabbed OpenMP in the back and I'm pissed, but their comments at the time were spot-on (OpenMP was moving so slow ... they just couldn't wait).
- The applications community failed themselves ...
 - If you don't commit to a standard and use "the next cool thing" you end up with the diversity of overlapping options we have today. Think about what happened with OpenMP and MPI.

No single processor is best at everything

- The idea that you should move everything to the GPU makes no sense
- **Heterogeneous Computing:** Run sub-problems in parallel on the hardware best suited to them.



OpenMP: the only option for mixing CPU and GPU programming in a single program

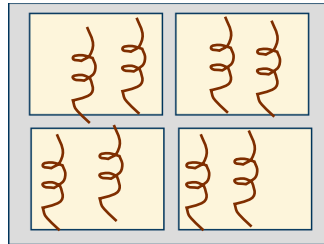
```
int main()
{
    float a[N], b[N], c[N], res[N];
    int err=0;

    // fill the arrays
    #pragma omp parallel for
    for (int i=0; i<N; i++){
        a[i] = (float)i;
        b[i] = 2.0*(float)i;
        c[i] = 0.0;
        res[i] = i + 2*i;
    }

    // add two vectors
    #pragma omp target
    #pragma omp loop
    for (int i=0; i<N; i++){
        c[i] = a[i] + b[i];
    }

    // test results
    #pragma omp parallel for reduction(+:err)
    for(int i=0; i<N; i++){
        float val = c[i] - res[i];
        val = val*val;
        if(val>TOL) err++;
    }
    printf("vectors added with %d errors\n", err);
    return 0;
}
```

Hardware is diverse ... and its only getting worse!!!



CPU

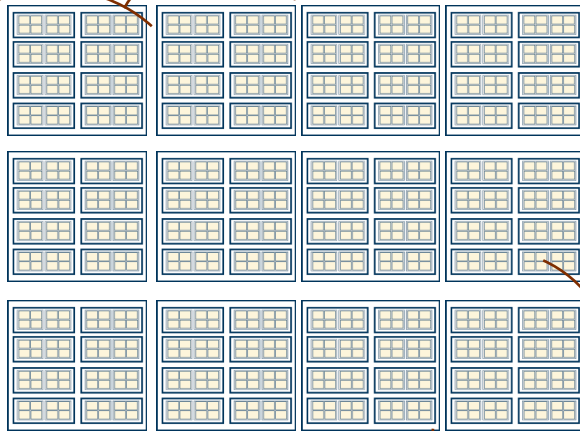


SIMD/Vector

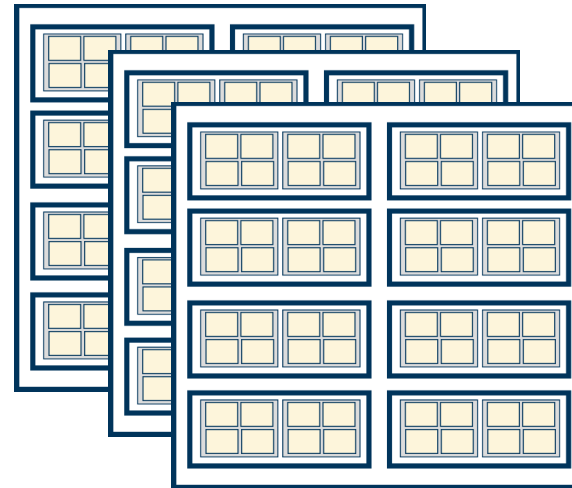


GPU

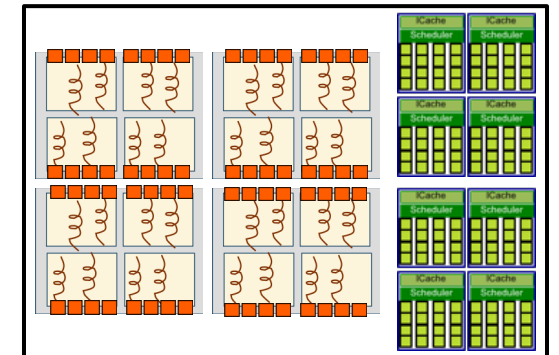
We'll now consider parallel programming with distributed memories



Cloud



Cluster



Heterogeneous node

Parallel API's: MPI ... the Message Passing Interface

MPI: An API for Writing Applications for Distributed memory systems

- A library of routines to coordinate the execution of multiple processes.
- Provides point to point and collective communication in Fortran, C and C++
- Unifies last 35+ years of cluster computing and MPP practice

MPI_Type_contiguous

MPI_Recv_init

MPI_Scan

MPI_Allgather

MPI_Start

MPI_Bsend_init

MPI_Sendrecv_replace

MPI_Alltoallv

MPI_Send

MPI_Bcast

MPI_Type_free

MPI_Unpack

CSOMP_ORDERED

MPI_Reduce

MPI_Bsend_init

MPI_Recv

MPI_Test_cancelled

MPI_Ssend

MPI_Waitall

MPI_Recv_init

MPI_Group_size

MPI_COMM_WORLD

MPI_Group_compare

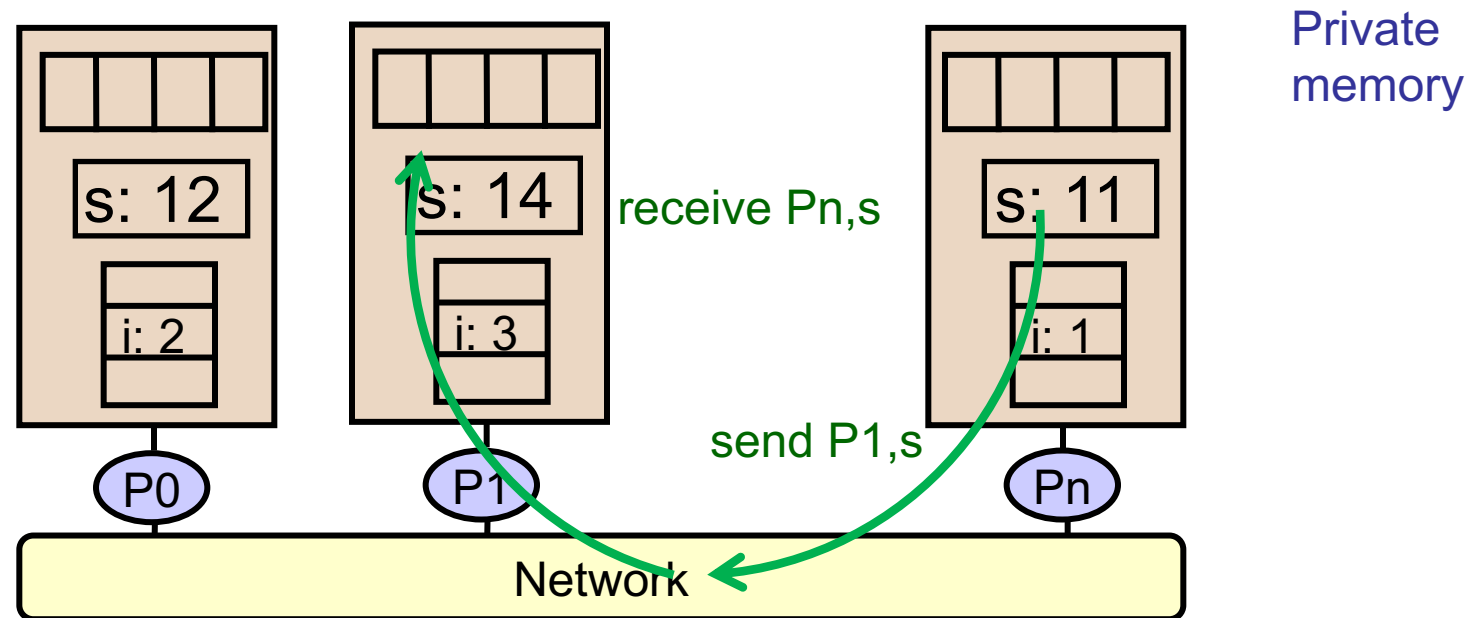
MPI_Startall

MPI_Pack

omp_set_lock(lck)

Programming Model: Message Passing

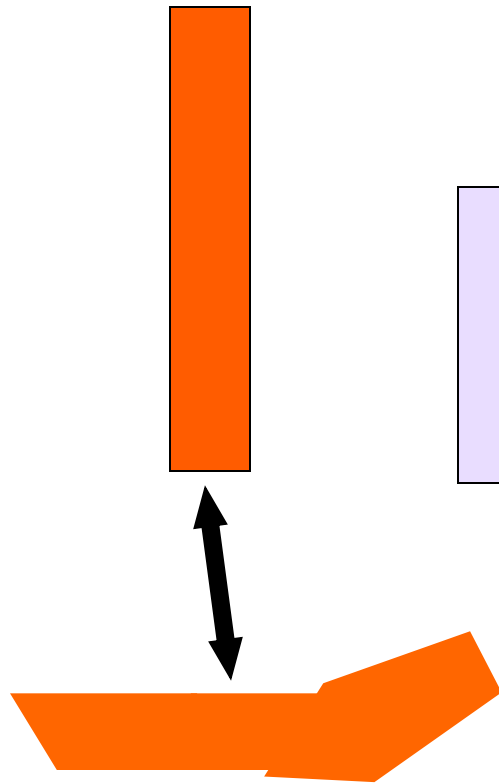
- Program consists of a collection of processes.
 - Number of processes almost always fixed at program startup time
 - Local address space per node -- NO physically shared memory.
 - Logically shared data is partitioned over local processes.
- Processes communicate by explicit send/receive pairs
 - Synchronization is implicit by communication events.
 - MPI (Message Passing Interface) is the most commonly used API



How do people use MPI?

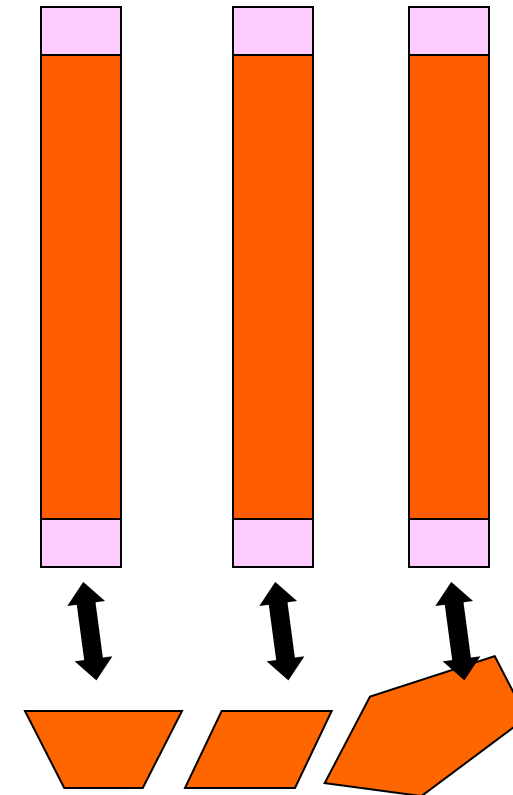
The SPMD Design Pattern

A sequential program
working on a data set



Replicate the program.
Add glue code
Break up the data

- A single program working on a decomposed data set.
- Use Node ID and numb of nodes to split up work between processes
- Coordination by passing messages.



MPI Hello World Program

```
#include <stdio.h>
#include <mpi.h>
int main (int argc, char **argv){
    int rank, size;
    MPI_Init (&argc, &argv);
    MPI_Comm_rank (MPI_COMM_WORLD, &rank);
    MPI_Comm_size (MPI_COMM_WORLD, &size);
    printf( "Hello from process %d of %d\n",
            rank, size );

    MPI_Finalize();
    return 0;
}
```

MPI Hello World Program

```
#include <stdio.h>
#include <mpi.h>
int main (int argc, char **argv){
    int rank, size;
    MPI_Init (&argc, &argv);
    MPI_Comm_rank (MPI_COMM_WORLD, &rank);
    MPI_Comm_size (MPI_COMM_WORLD, &size);
    printf( "Hello from process %d of %d\n",
           rank, size );

    MPI_Finalize();
    return 0;
}
```

int MPI_Init (int* argc, char* argv[])

- Initializes the MPI library ... called before any other MPI functions.
- argc and argv are the command line args passed from main()

int MPI_Finalize (void)

- Frees memory allocated by the MPI library ... close every MPI program with a call to MPI_Finalize

MPI Hello World Program

```
int MPI_Comm_size (MPI_Comm comm, int* size)
```

- `MPI_Comm`, an *opaque data type called a communicator*.
Default context: `MPI_COMM_WORLD` (all processes)
- `MPI_Comm_size` returns the number of processes in the process group associated with the communicator

```
#include <stdio.h>
#include <mpi.h>
int main (int argc, char **argv){
    int rank, size;
    MPI_Init (&argc, &argv);
    MPI_Comm_rank (MPI_COMM_WORLD, &rank);
    MPI_Comm_size (MPI_COMM_WORLD, &size);
    printf( "Hello from process %d of %d\n",
            rank, size );

    MPI_Finalize();
    return 0;
}
```

Communicators consist of two parts, a **context** and a **process group**.

The communicator lets one control how groups of messages interact.

Communicators support modular SW ... i.e. I can give a library module its own communicator and know that its messages can't collide with messages originating from outside the module

MPI Hello World Program

```
int MPI_Comm_rank (MPI_Comm comm, int* rank)
```

- `MPI_Comm`, an *opaque data type*, a communicator. Default context: `MPI_COMM_WORLD` (all processes)
- `MPI_Comm_rank` An integer ranging from 0 to “(num of procs)-1”

```
#include <stdio.h>
#include <mpi.h>
int main (int argc, char **argv){
    int rank, size;
    MPI_Init (&argc, &argv);
    MPI_Comm_rank (MPI_COMM_WORLD, &rank);
    MPI_Comm_size (MPI_COMM_WORLD, &size);
    printf( "Hello from process %d of %d\n",
            rank, size );

    MPI_Finalize();
    return 0;
}
```

Note that other than `init()` and `finalize()`, every MPI function has a communicator.

This makes sense .. You need a context and group of processes that the MPI functions impact ... and those come from the communicator.

Bulk Synchronous Programming (BSP):

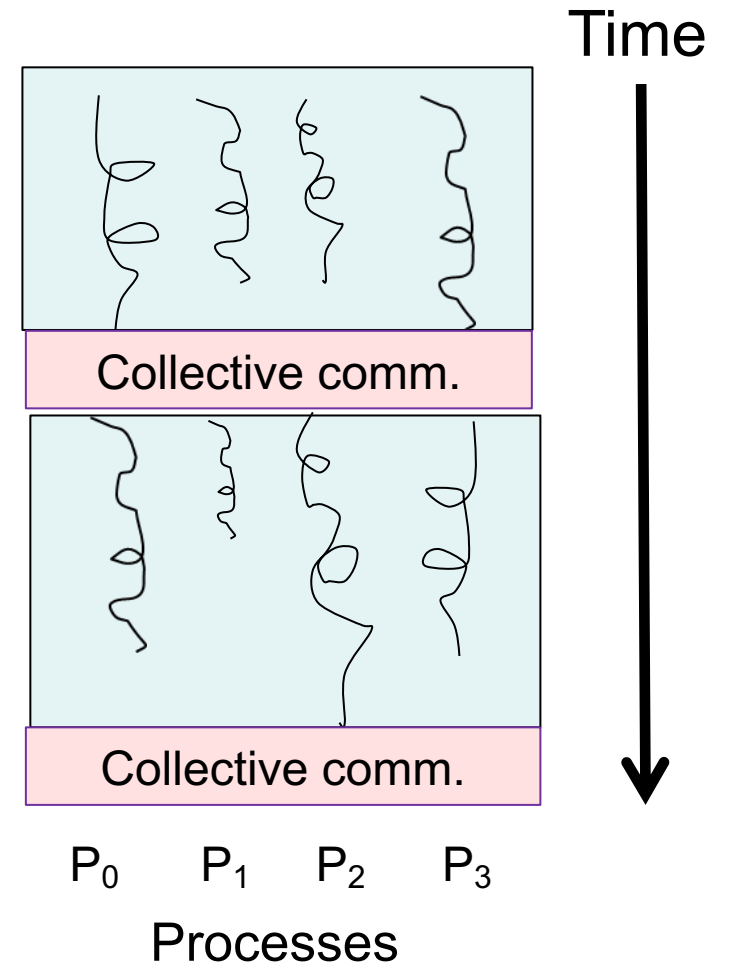
A common design pattern used with MPI Programs

BSP includes the map-reduce pattern commonly used in the cloud. It's easy to use and surprisingly useful

- Many MPI applications have few (if any) sends and receives.

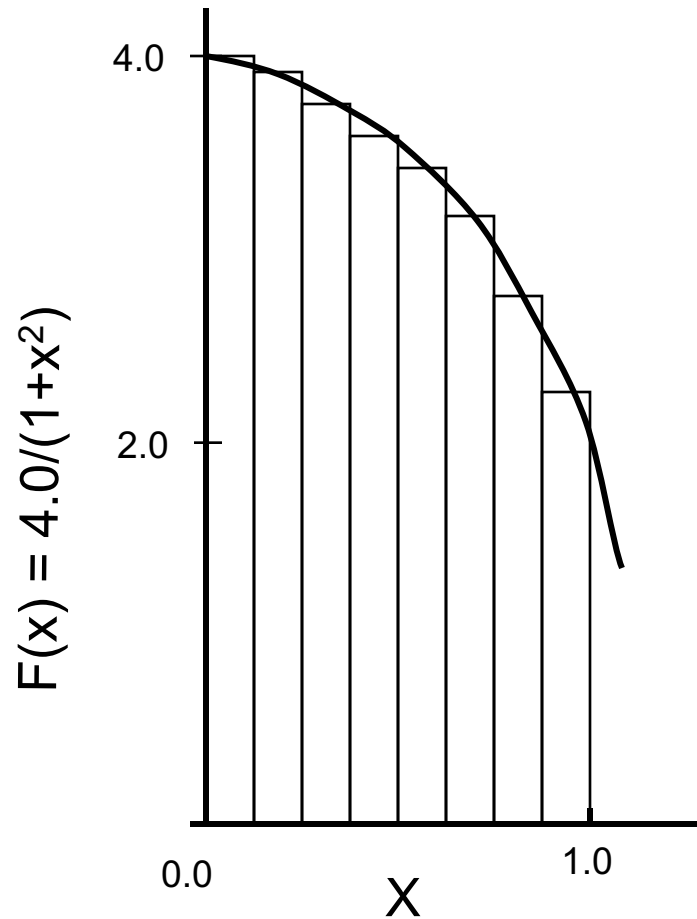
They use the following very common pattern:

- Use the Single Program Multiple Data pattern
- Each process maintains a local view of the global data
- A problem broken down into phases each of which is composed of two subphases:
 - Compute on local view of data
 - Communicate to update global view on all processes (collective communication).
- Continue phases until complete



This is a subset of the SPMD pattern sometimes referred to as the Bulk Synchronous pattern.

Example Problem: Numerical Integration



Mathematically, we know that:

$$\int_0^1 \frac{4.0}{(1+x^2)} dx = \pi$$

We can approximate the integral as a sum of rectangles:

$$\sum_{i=0}^N F(x_i) \Delta x \approx \pi$$

Where each rectangle has width Δx and height $F(x_i)$ at the middle of interval i .

PI Program: an example

```
static long num_steps = 100000;
double step;
void main ()
{   int i;   double x, pi, sum = 0.0;

    step = 1.0/(double) num_steps;
    x = 0.5 * step;
    for (i=0;i<= num_steps; i++){
        x+=step;
        sum += 4.0/(1.0+x*x);
    }
    pi = step * sum;
}
```

Pi program in MPI

```
#include <mpi.h>
```

```
void main (int argc, char *argv[])
```

```
{
```

```
    int i, my_id, numprocs; double x, pi, step, sum = 0.0 ;
```

```
    step = 1.0/(double) num_steps ;
```

```
    MPI_Init(&argc, &argv) ;
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &my_id) ;
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &numprocs) ;
```

```
    my_steps = num_steps/numprocs ;
```

```
    for (i=my_id; i<num_steps; i+=numprocs)
```

```
    {
```

```
        x = (i+0.5)*step;
```

```
        sum += 4.0/(1.0+x*x);
```

```
    }
```

```
    sum *= step ;
```

```
    MPI_Reduce(&sum, &pi, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD); ;
```

```
}
```



Sum values in “sum” from
each process and place it
in “pi” on process 0

MPI Pi program performance (on my laptop)

```
#include <mpi.h>
```

```
void main (int argc, char *argv[])
```

```
{
```

```
    int i, my_id, numprocs; double x, pi, step, sum = 0.0 ;
```

```
    step = 1.0/(double) num_steps ;
```

```
    MPI_Init(&argc, &argv) ;
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &my_id) ;
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &numprocs) ;
```

```
    double init_time = MPI_Wtime();
```

```
    my_steps = num_steps/numprocs ;
```

```
    for (i=my_id*my_steps; i<(my_id+1)*my_steps ; i++)
```

```
    {
```

```
        x = (i+0.5)*step;
```

```
        sum += 4.0/(1.0+x*x);
```

```
    }
```

```
    sum *= step ;
```

```
    MPI_Reduce(&sum, &pi, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
```

```
    if(my_id == 0) printf(" runtime = %lf\n",MPI_Wtime()-init_time);
```

```
}
```

Thread or procs	OpenMP SPMD critical	OpenMP PI Loop	MPI
1	0.85	0.43	0.84
2	0.48	0.23	0.48
3	0.47	0.23	0.46
4	0.46	0.23	0.46

*Intel compiler (icpc) with -O3 on Apple OS X 10.7.3 with a dual core (four HW thread)
Intel® Core™ i5 processor at 1.7 Ghz and 4 Gbyte DDR3 memory at 1.333 Ghz.

Message passing

What message passing looks like in "real" programs

Example: finite difference methods

- Solve the heat diffusion equation in 1 D:
 - $u(x,t)$ describes the temperature field
 - We set the heat diffusion constant to one
 - Boundary conditions, constant u at endpoints.

$$\frac{\partial^2 u}{\partial x^2} = \frac{\partial u}{\partial t}$$

- map onto a mesh with stepsize h and k

$$x_i = x_0 + ih \quad t_i = t_0 + ik$$

- Central difference approximation for spatial derivative (at fixed time)

$$\frac{\partial^2 u}{\partial x^2} = \frac{u_{j+1} - 2u_j + u_{j-1}}{h^2}$$

- Time derivative at $t = t^{n+1}$

$$\frac{du}{dt} = \frac{u^{n+1} - u^n}{k}$$

Example: Explicit finite differences

- Combining time derivative expression using spatial derivative at $t = t_n$

$$\frac{u_j^{n+1} - u_j^n}{k} = \frac{u_{j+1}^n - 2u_j^n + u_{j-1}^n}{h^2}$$

- Solve for u at time $n+1$ and step j

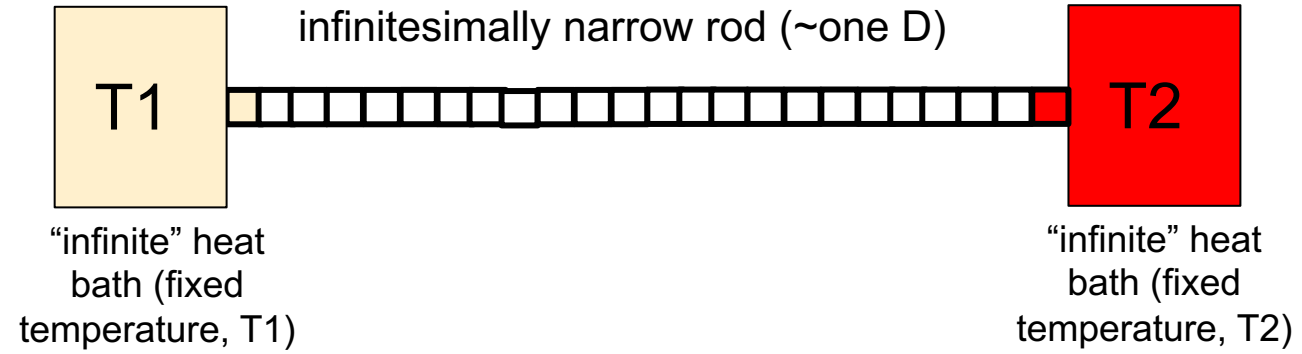
$$u_j^{n+1} = (1 - 2r)u_j^n + ru_{j-1}^n + ru_{j+1}^n \quad r = k/h^2$$

- The solution at $t = t_{n+1}$ is determined explicitly from the solution at $t = t_n$ (assume $u[t][0] = u[t][N] = \text{Constant}$ for all t).

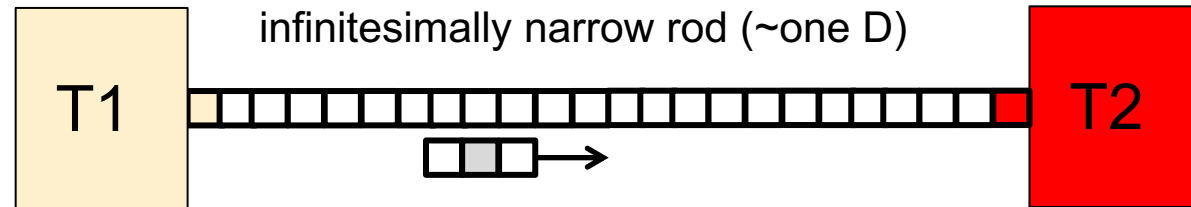
```
for (int t = 0; t < N_STEPS-1; ++t)
    for (int x = 1; x < N-1; ++x)
        u[t+1][x] = u[t][x] + r*(u[t][x+1] - 2*u[t][x] + u[t][x-1]);
```

- Explicit methods are easy to compute ... each point updated based on nearest neighbors. Converges for $r < 1/2$.

Heat Diffusion equation



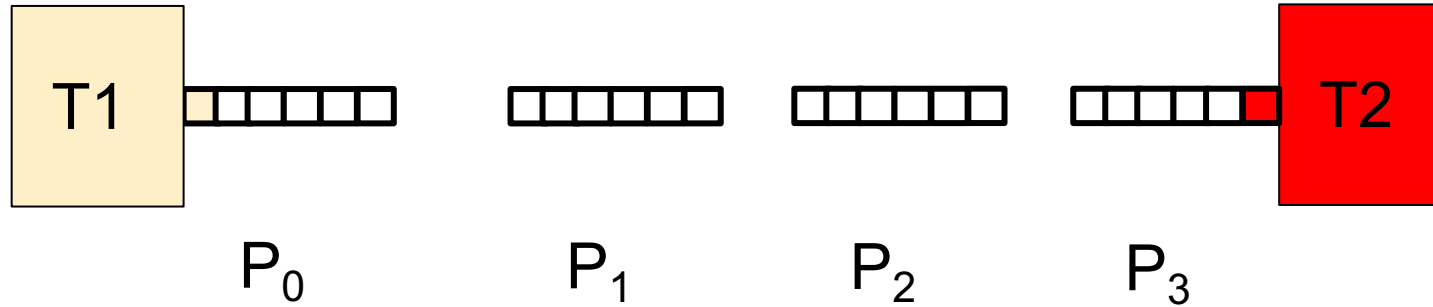
Heat Diffusion equation



Pictorially, you are sliding a three point "stencil" across the domain (u) and updating the center point at each stop.

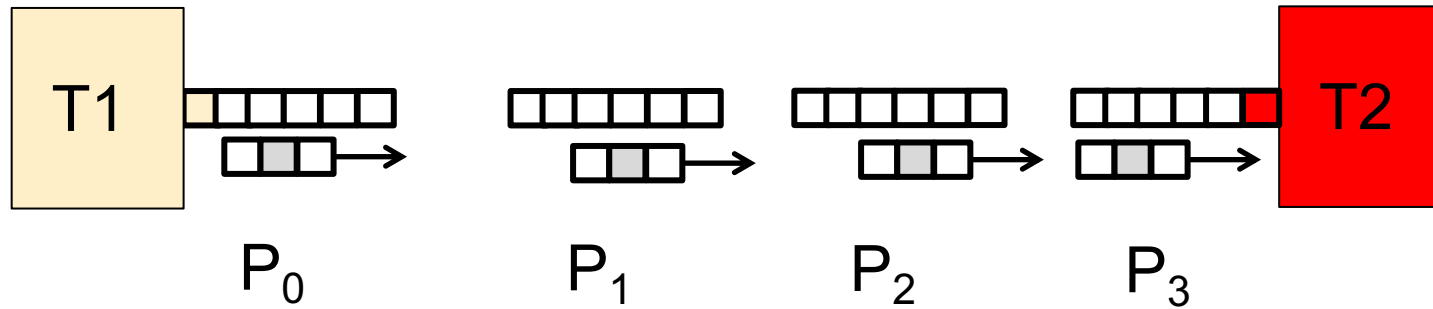
Heat Diffusion equation: in parallel

- Break it into chunks assigning one chunk to each process.



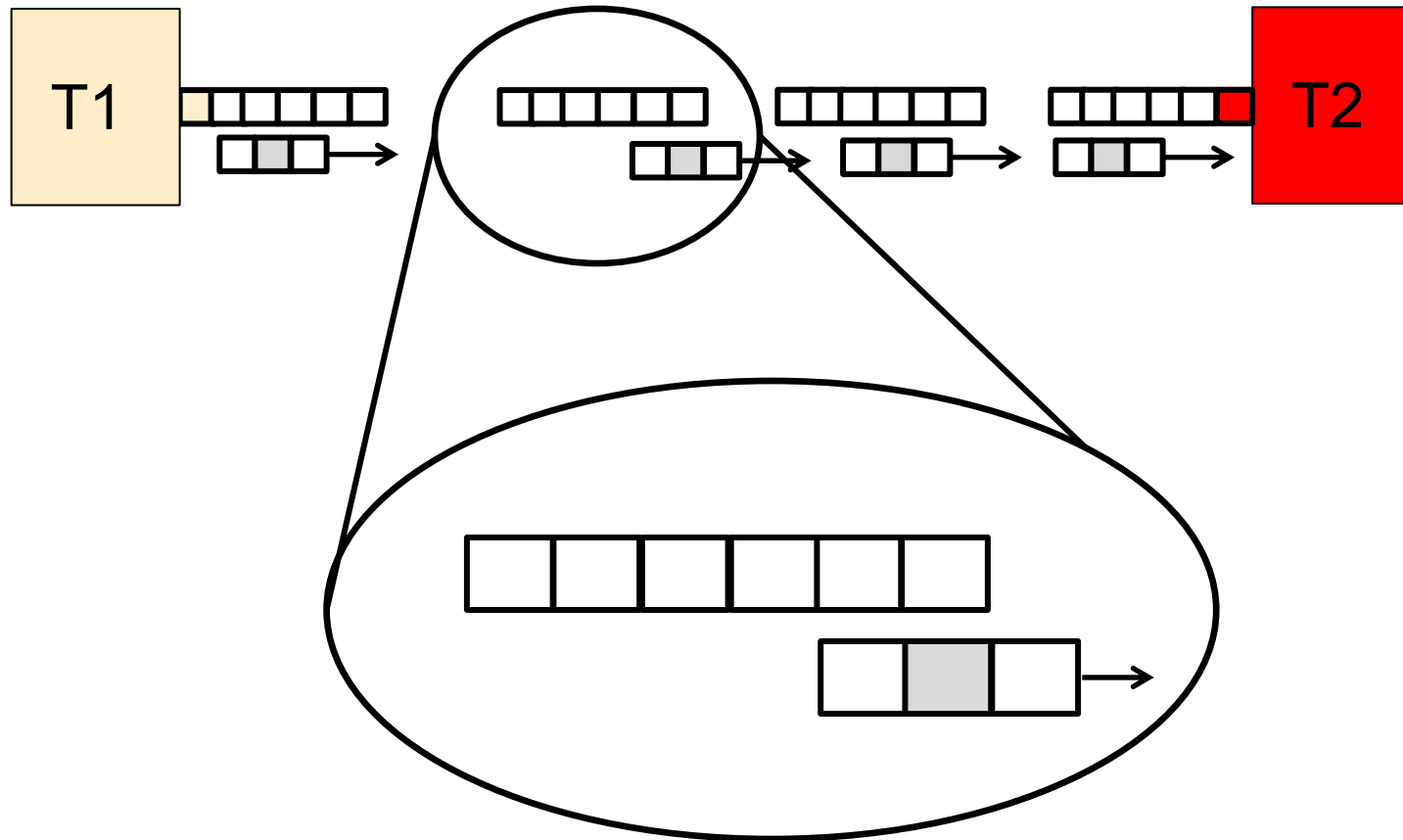
Heat Diffusion equation

- Each process works on its own chunk ... sliding the stencil across the domain to updates its own data.



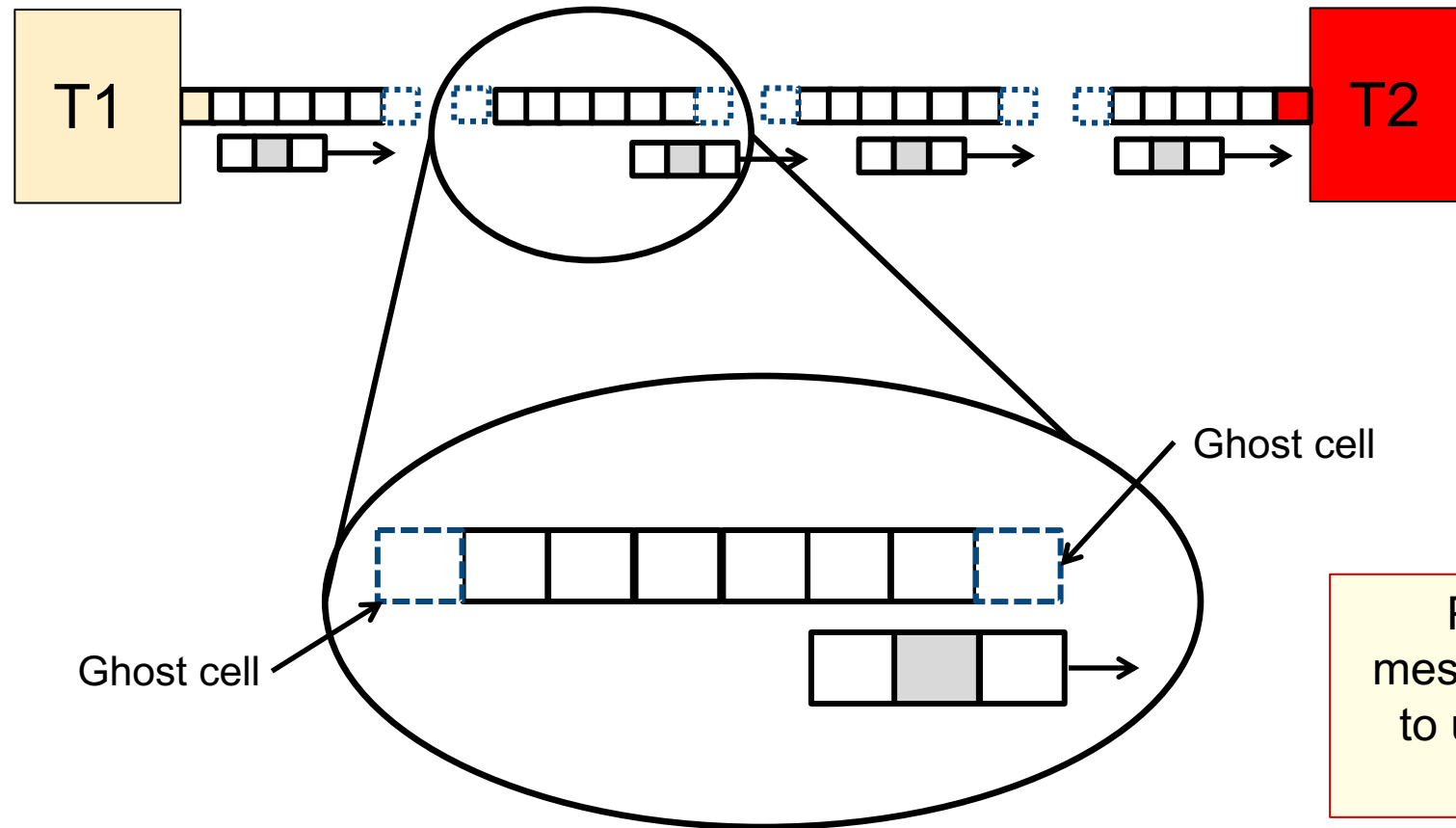
Heat Diffusion equation

- What about the ends of each chunk ... where the stencil runs off the end and has missing values for the computation?



Heat Diffusion equation

- We add ghost cells to the ends of each chunk, update them with the required values from neighbor chunks at each time step ... hence giving the stencil everything it needs on any given chunk to update all of its values.



Processes send messages to neighbors to update ghost cells after each step

Heat Diffusion MPI Example

```
MPI_Init (&argc, &argv);
MPI_Comm_size (MPI_COMM_WORLD, &P);
MPI_Comm_rank (MPI_COMM_WORLD, &myID);
double *u      = malloc (sizeof(double) * (2 + N/P)) // include "Ghost Cells" to hold
double *up1 = malloc (sizeof(double) * (2 + N/P)); // values from my neighbors

initialize_data(uk, ukp1, N, P);
for (int t = 0; t < N_STEPS; ++t){
    if (myID != 0) MPI_Send (&u[1], 1, MPI_DOUBLE, myID-1, 0, MPI_COMM_WORLD);
    if (myID != P-1) MPI_Recv (&u[N/P+1], 1, MPI_DOUBLE, myID+1, 0, MPI_COMM_WORLD, &status);
    if (myID != P-1) MPI_Send (&u[N/P], 1, MPI_DOUBLE, myID+1, 0, MPI_COMM_WORLD);
    if (myID != 0) MPI_Recv (&u[0], 1, MPI_DOUBLE, myID-1, 0, MPI_COMM_WORLD, &status);

    for (int x = 1; x <= N/P; ++x)
        up1[x] = u[x] + (k / (h*h)) * (u[x+1] - 2*u[x] + u[x-1]);
    if (myID != 0)
        up1[1] = u[1] + (k / (h*h)) * (u[1+1] - 2*u[1] + u[1-1]);
    if (myID != P-1)
        up1[N/P] = u[N/P] + (k/(h*h)) * (u[N/P+1] - 2*u[N/P] + u[N/P-1]);
    temp = up1; up1 = u; u = temp;
} // End of for (int t ...) loop

MPI_Finalize();
return 0;
```

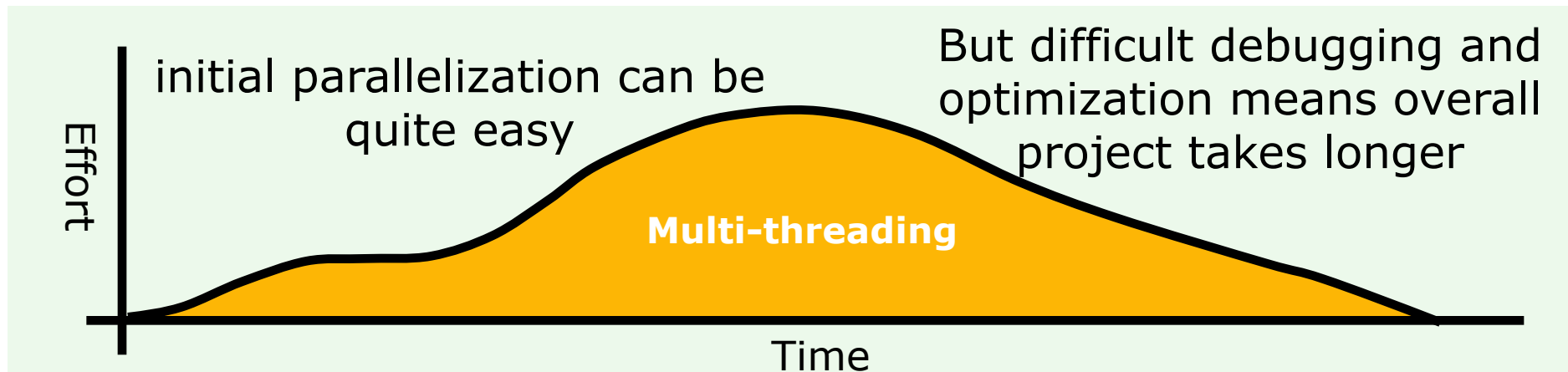
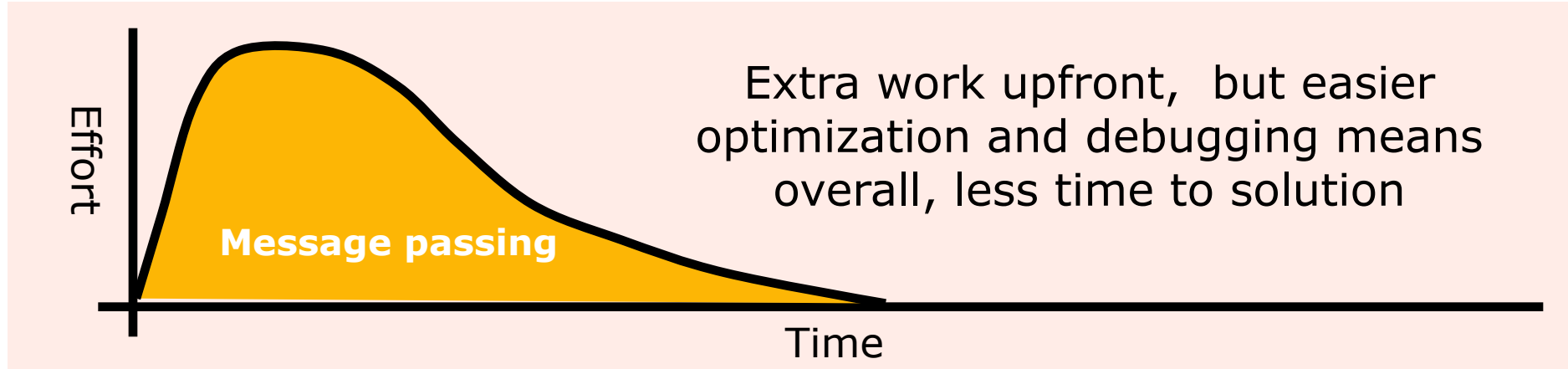
**Exchange ghost
cells**

**Do the
computation, but
keep track of
boundary
conditions**

The code can get a bit ugly as each process figures out who to send to and who to receive from.

... but the concept is straightforward and eventually you get used to it.

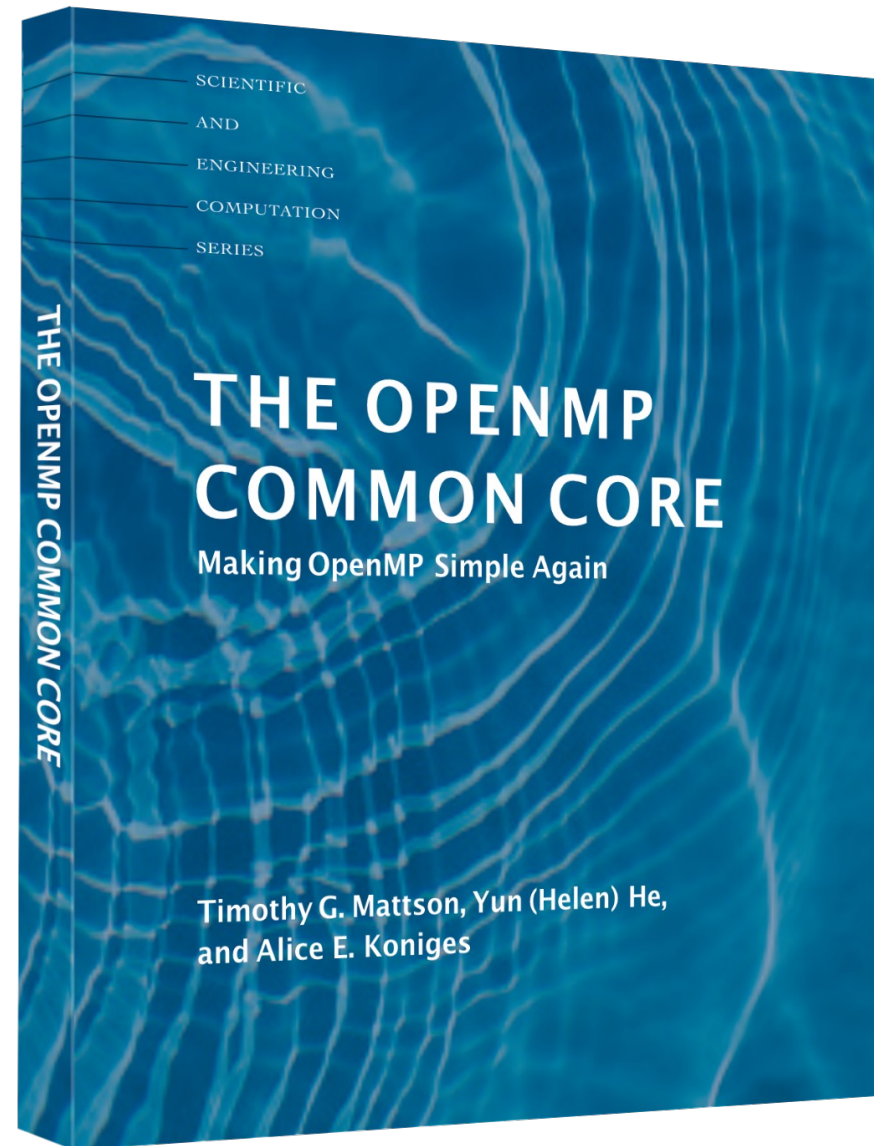
Does a shared address space make programming easier?



Proving that a shared address space program using semaphores is race free is an NP-complete problem*

To learn OpenMP:

- An exciting new book that Covers the Common Core of OpenMP plus a few key features beyond the common core that people frequently use
- It's geared towards people learning OpenMP, but as one commentator put it ... **everyone at any skill level should read the memory model chapters.**
- Available from MIT Press



Books about OpenMP

A great book that covers
OpenMP features beyond
OpenMP 2.5

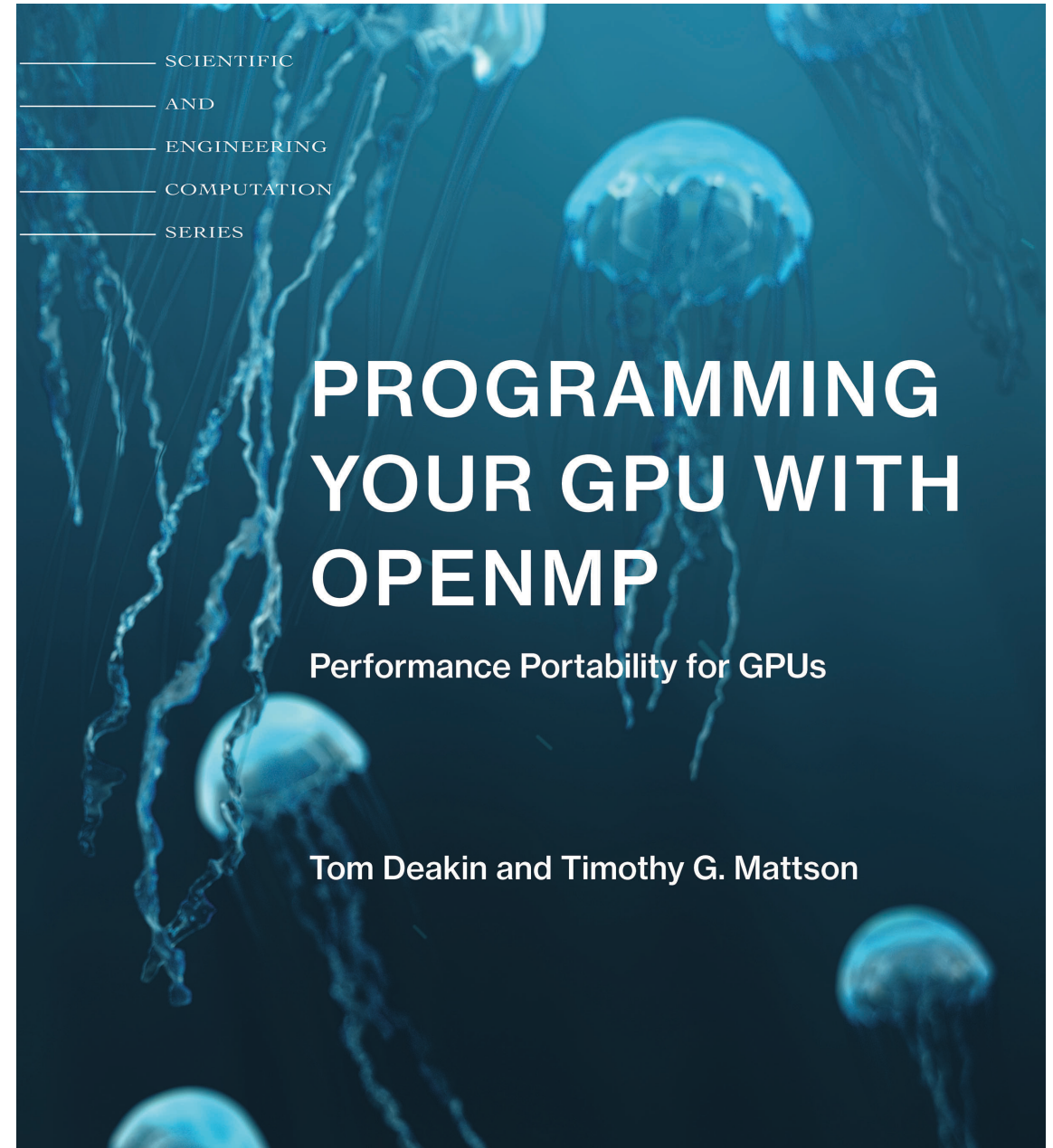


Books about OpenMP

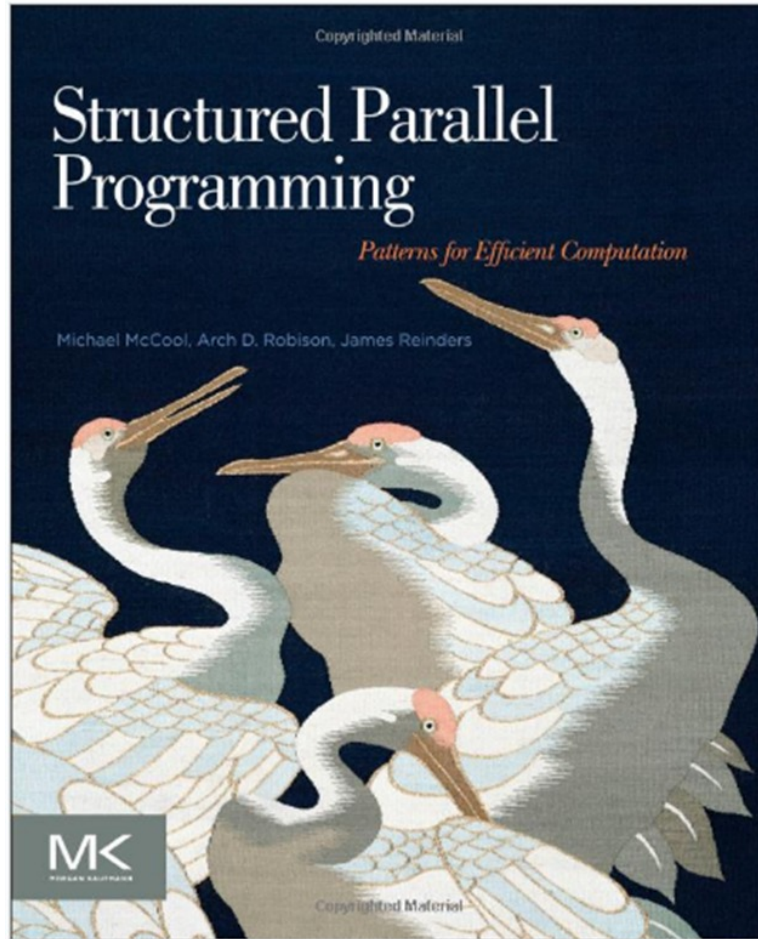
The latest book on OpenMP ...

Released in November 2023.

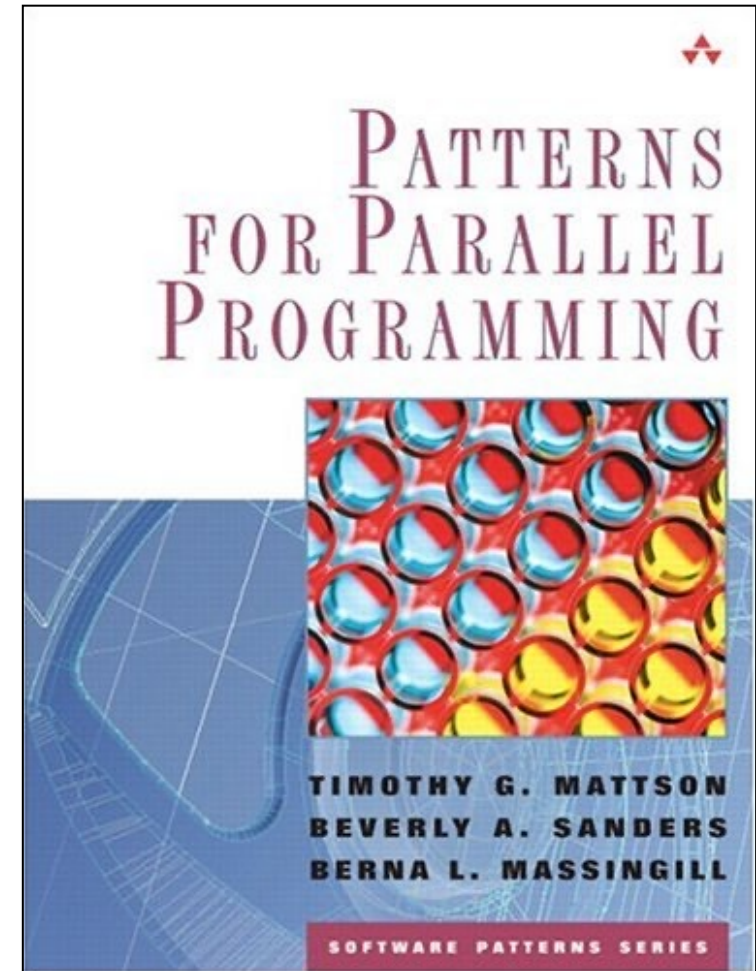
A book about how to use OpenMP to
program a GPU.



Background references



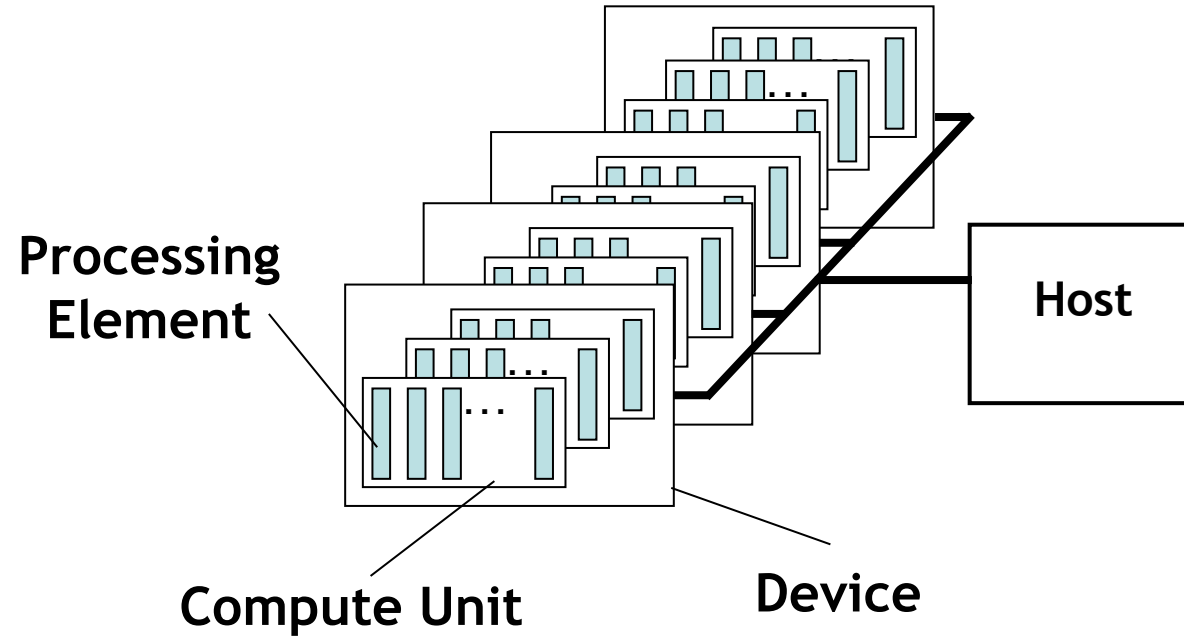
A great book that explores key patterns with Cilk, TBB, OpenCL, and OpenMP (by McCool, Robison, and Reinders)



- A book about how to “think parallel” with examples in OpenMP, MPI and java

**Extra Content: A detailed tutorial to
programming your GPU with OpenMP**

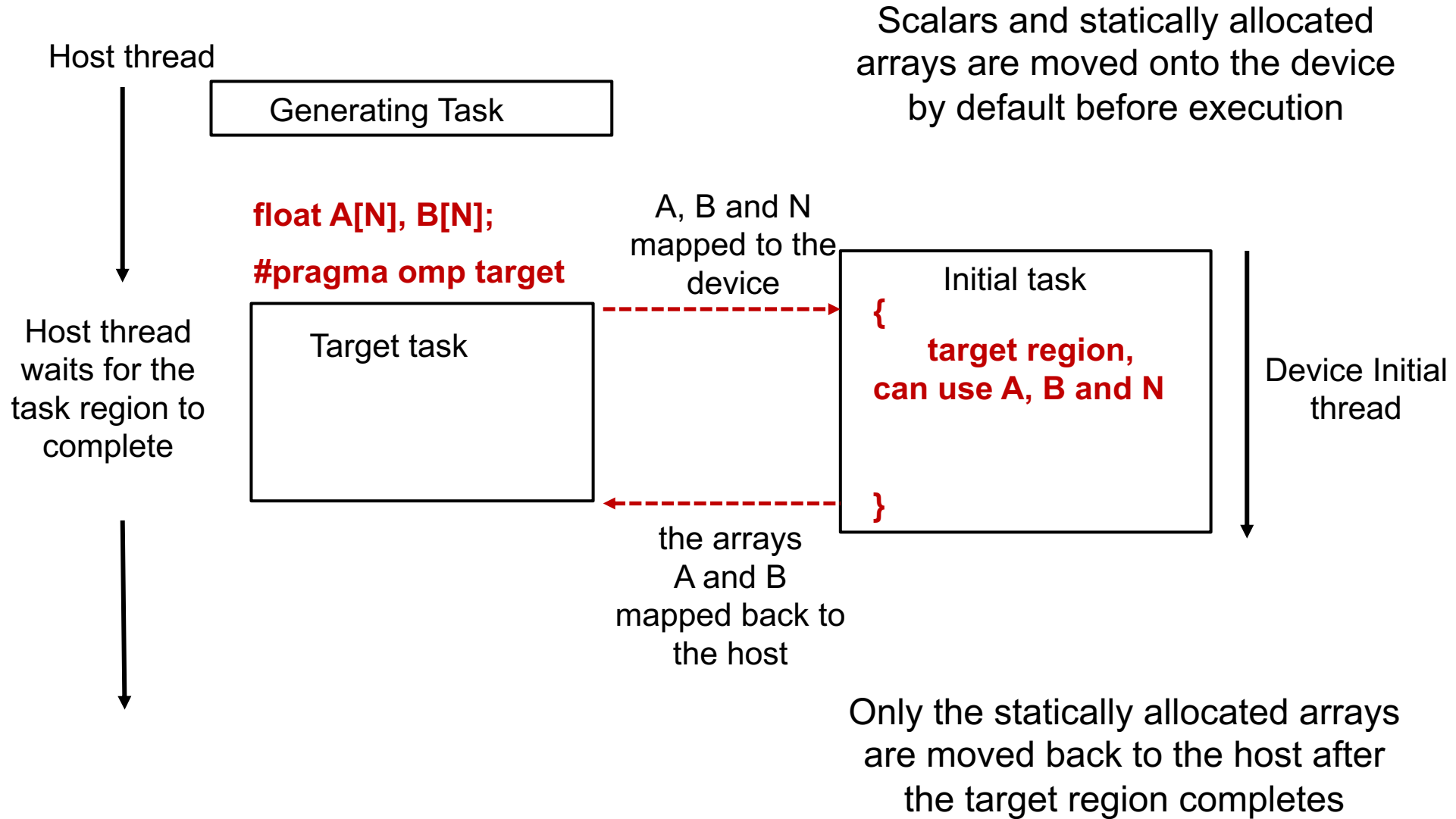
A Generic Host/Device Platform Model



- One **Host** and one or more **Devices**
 - Each Device is composed of one or more Compute Units
 - Each Compute Unit is divided into one or more **Processing Elements**
- Memory divided into **host memory** and **device memory**

Running code on the GPU:

The target construct and default data movement



Default Data Sharing: example

```
int main(void) {  
    int N = 1024;  
    double A[N], B[N];
```

```
#pragma omp target  
{
```

```
    for (int ii = 0; ii < N; ++ii) {
```

```
        A[ii] = A[ii] + B[ii];
```

```
    }
```

```
} // end of target region
```

```
}
```

1. Variables created in host memory.

2. Scalar **N** and stack arrays **A** and **B** are copied to device memory. Execution transferred to device.

3. **ii** is **private** on the device as it's declared within the target region

4. Execution on the device.

5. stack arrays **A** and **B** are copied *from* device memory back to the host. Host resumes execution.

Now let's run code in parallel on the device

```
int main(void) {  
    int N = 1024;  
    double A[N], B[N];  
  
    #pragma omp target  
    {  
        #pragma omp loop  
        for (int ii = 0; ii < N; ++ii) {  
  
            A[ii] = A[ii] + B[ii];  
  
        }  
  
    } // end of target region  
}
```

The loop construct tells the compiler:

"this loop will execute correctly if the loop iterations run in any order. You can safely run them concurrently. And the loop-body doesn't contain any OpenMP constructs. So do whatever you can to make the code run fast"

The loop construct is a declarative construct. You tell the compiler what you want done but you DO NOT tell it how to "do it". This is new for OpenMP

Exercise: Parallel vector addition on a GPU

- Make a copy of the vadd.c program in OMP_GPU_Exercises

- vadd.c Adds together two arrays, element by element:

```
for(i=0;i<N;i++) c[i]=a[i]+b[i];
```

- Parallelize your vadd program for a GPU.

- module load nvhpc/24.5
 - nvc -O3 -mp=gpu vadd.c

- Time it for large N and save the result. How does it compare to the CPU version?

- double omp_get_wtime();
 - #pragma omp target
 - #pragma omp loop
 - sbatch gpu_job.sh

For tiny little programs, OpenMP may opt to run the code on the host. You can force the OpenMP runtime to use the GPU by setting the OMP_TARGET_OFFLOAD environment variable

```
> OMP_TARGET_OFFLOAD=MANDATORY ./a.out
```

```
#!/bin/bash
#SBATCH -p sched_mit_psfc_gpu_r8
#SBATCH --reservation=cpsfr_2025
#SBATCH -o gpu-job.log-%j
#SBATCH --gres=gpu:1
module load nvhpc/24.5
OMP_TARGET_OFFLOAD=MANDATORY ./a.out
```

gpu_job.sh

```
salloc -p sched_mit_psfc_gpu_r8 --reservation=cpsfr_2025 --time=01:00:00
```

```
salloc -p sched_mit_psfc_gpu_r8 --reservation=cpsfr_2025 --gres=gpu:1 --time=01:00:00
```

git clone <https://github.com/tgmattso/ParProgForPhys.git>

Solution: Simple vector add in OpenMP on GPU

```
int main()
{
    float a[N], b[N], c[N], res[N];
    int err=0;

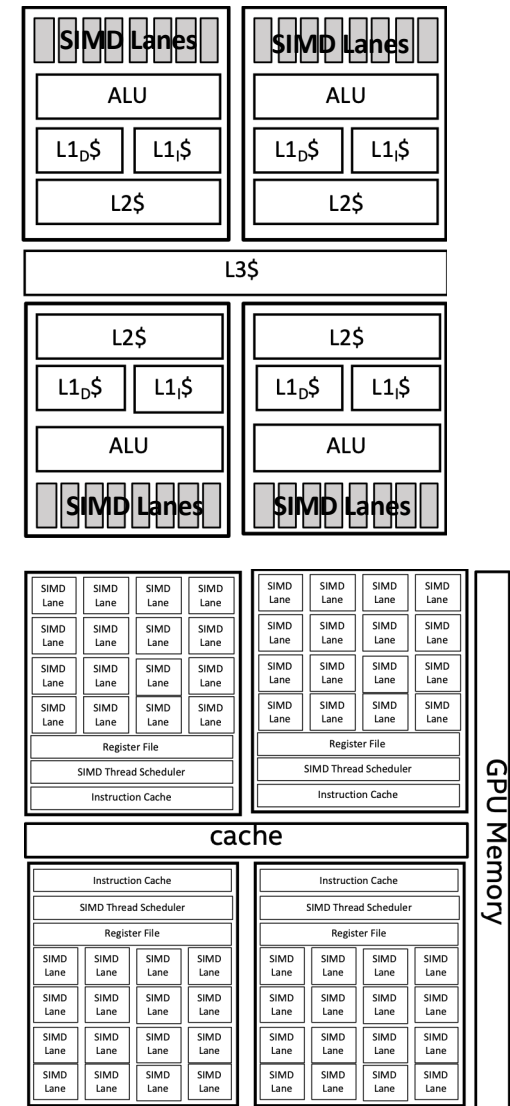
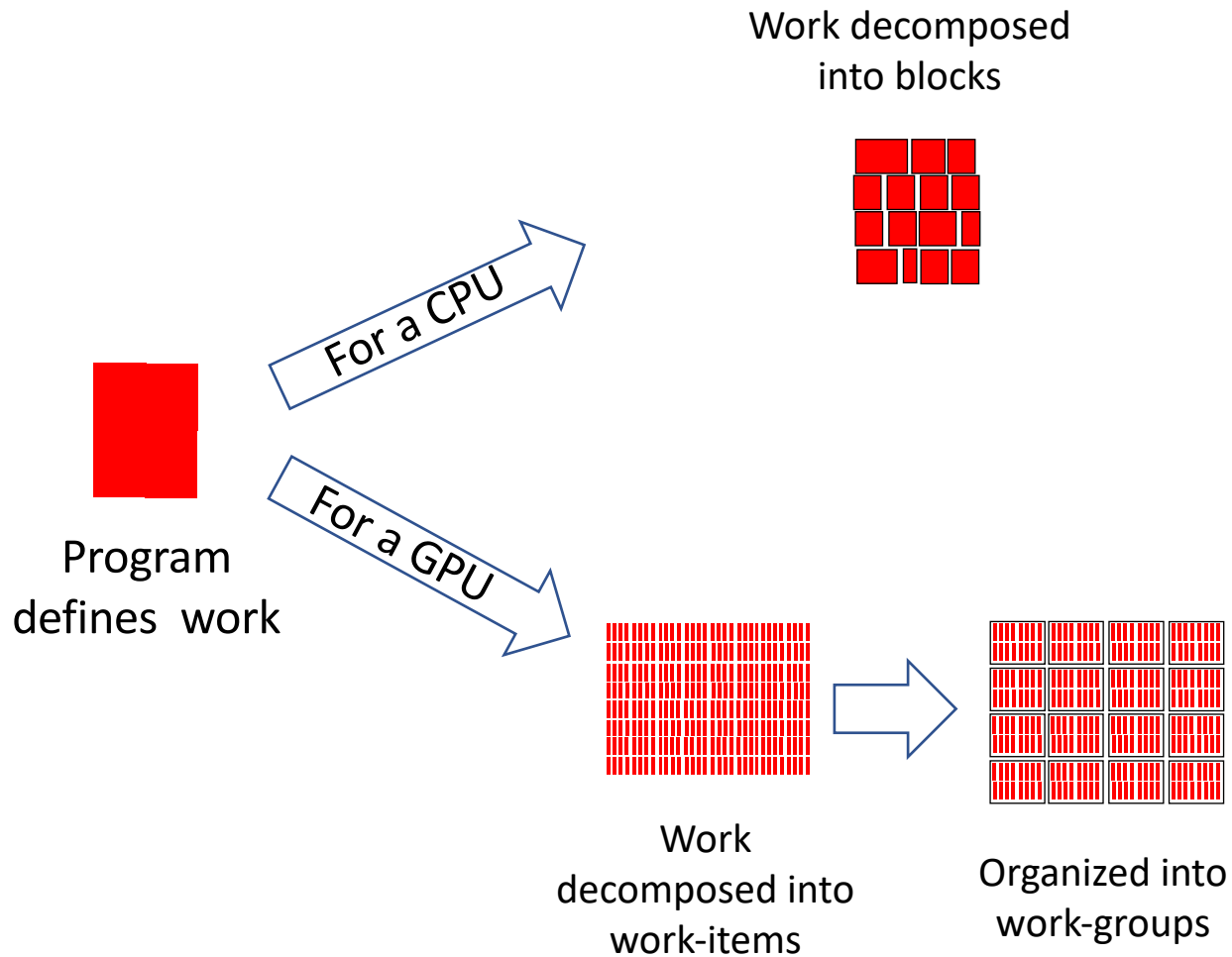
    // fill the arrays
    #pragma omp parallel for
    for (int i=0; i<N; i++){
        a[i] = (float)i;
        b[i] = 2.0*(float)i;
        c[i] = 0.0;
        res[i] = i + 2*i;
    }

    // add two vectors
    #pragma omp target
    #pragma omp loop
    for (int i=0; i<N; i++){
        c[i] = a[i] + b[i];
    }

    // test results
    #pragma omp parallel for reduction(+:err)
    for(int i=0; i<N; i++){
        float val = c[i] - res[i];
        val = val*val;
        if(val>TOL) err++;
    }
    printf("vectors added with %d errors\n", err);
    return 0;
}
```

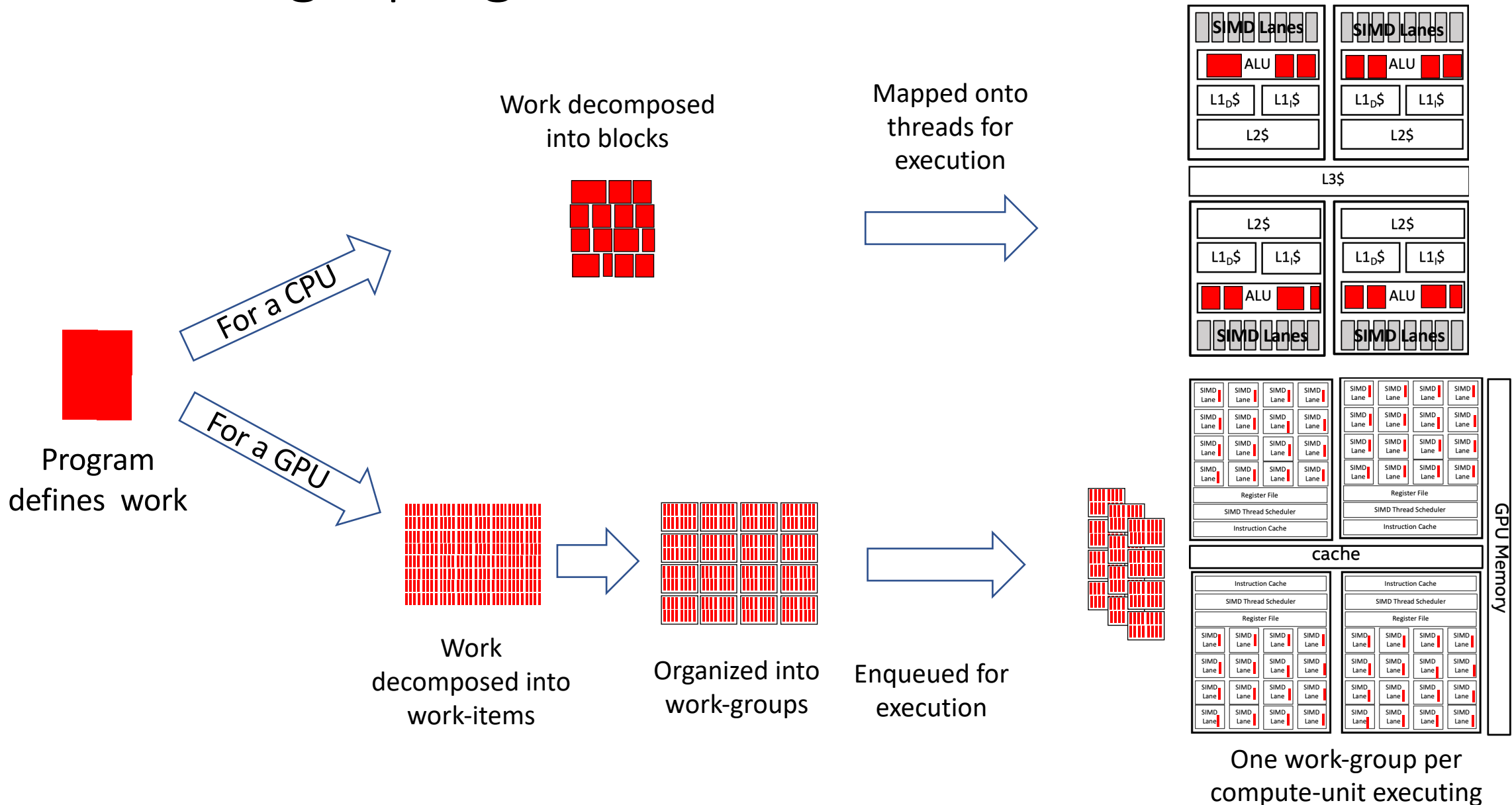
**Let's compare/contrast concurrency on a
CPU and a GPU**

Executing a program on CPUs and GPUs



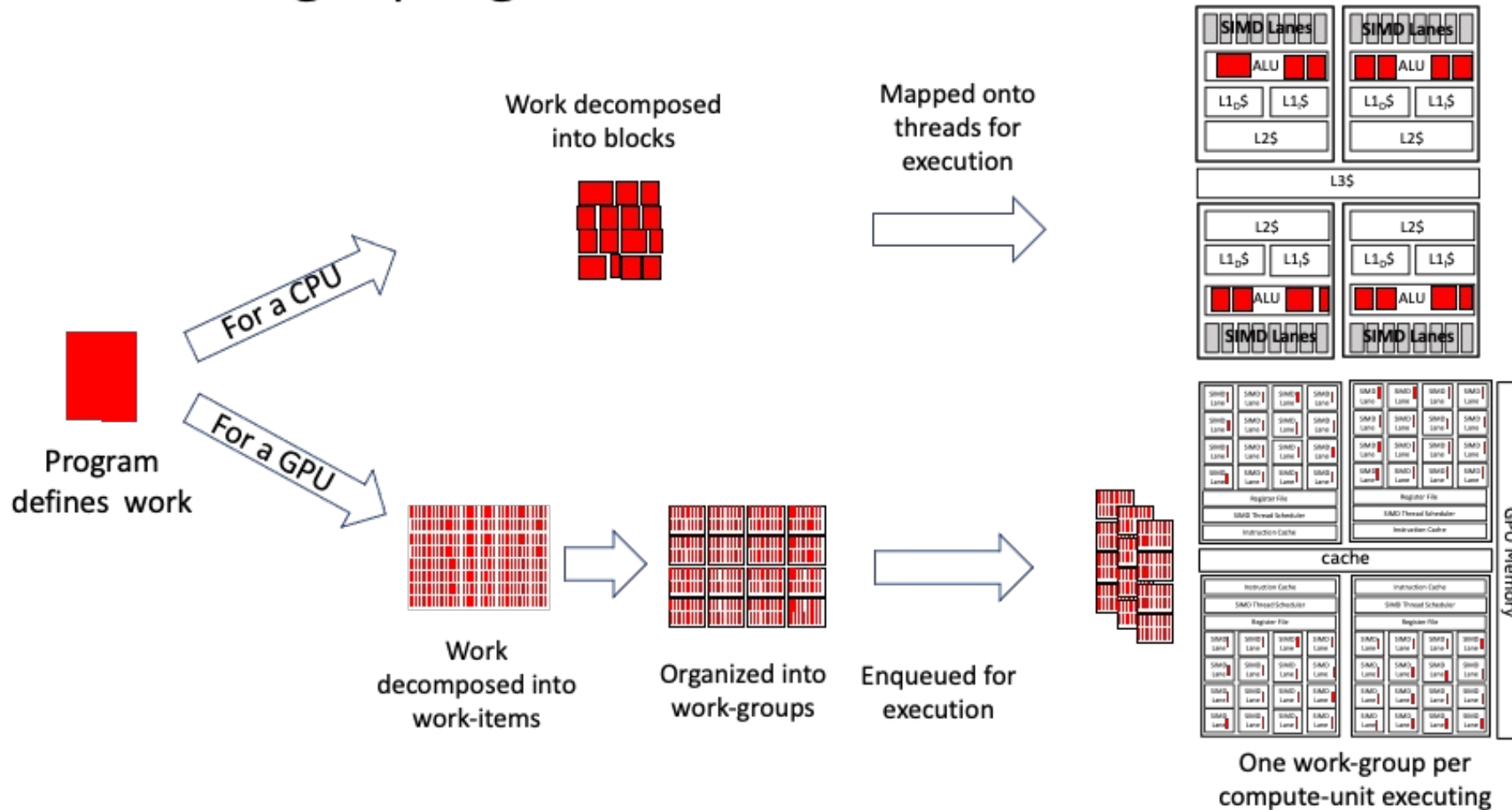
One work-group per
compute-unit executing

Executing a program on CPUs and GPUs



CPU/GPU execution modes

Executing a program on CPUs and GPUs

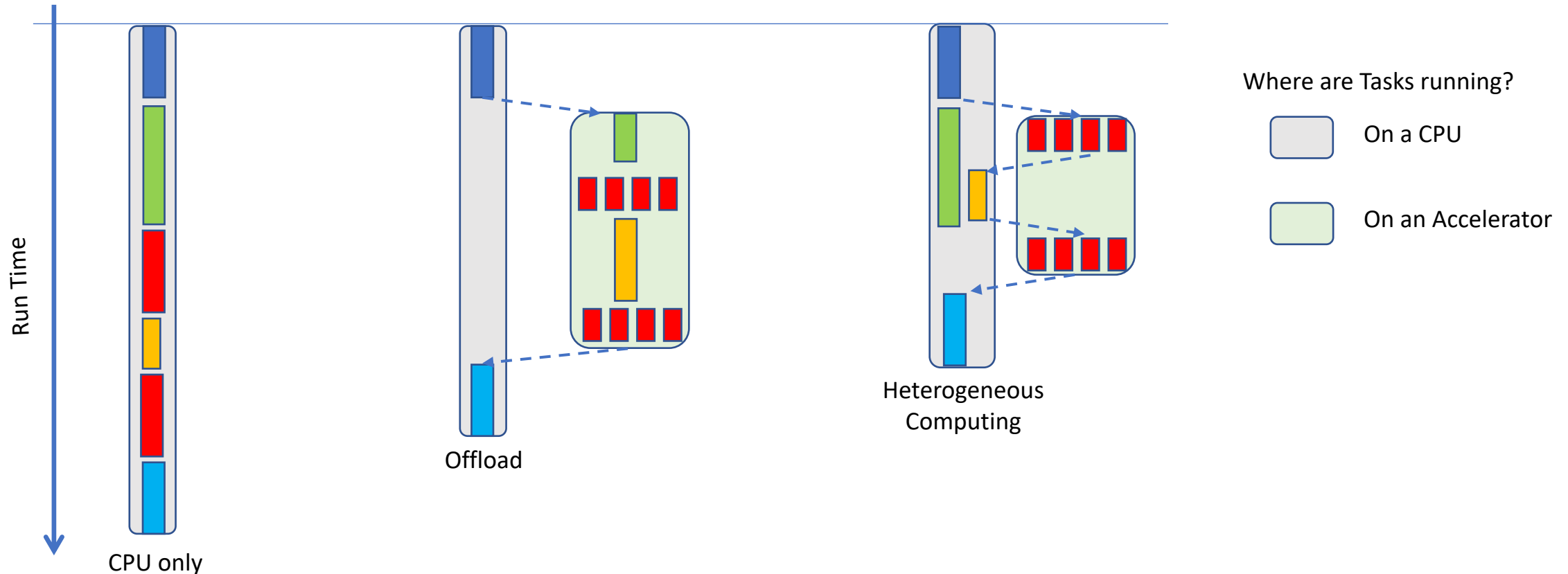


For a CPU, the threads are all active and able to make forward progress.

For a GPU, any given work-group might be in the queue waiting to execute.

No single processor is best at everything

- The idea that you should move everything to the GPU makes no sense
- **Heterogeneous Computing:** Run sub-problems in parallel on the hardware best suited to them.



Implicit data movement covers a small subset of the cases you need in a real program.

To be more general ... we need to manage data movement explicitly

Explicit data movement

- Previously, we described the rules for *implicit* data movement.
- We can *explicitly* control the movement of data using the **map** clause.
- Data allocated on the heap needs to be explicitly copied to/from the device:

```
int main(void) {  
    int ii=0, N = 1024;  
    int* A = (int *)malloc(sizeof(int)*N);
```

```
    #pragma omp target
```

```
    {  
        // N, ii and A all exist here  
        // The data that A points to (*A , A[ii]) DOES NOT exist here!  
    }  
}
```

Moving data with the map clause

```
int main(void) {  
    int N = 1024;  
    int* A = malloc(sizeof(int)*N);  
  
    #pragma omp target map(A[0:N])  
    {  
        // N, ii and A all exist here  
        // The data that A points to DOES exist here!  
    }  
}
```

Default mapping
map(tofrom: A[0:N])

Copy at start and end of
target region.

Controlling data movement

```
int i, a[N], b[N], c[N];
```

```
#pragma omp target map(to:a,b) map(tofrom:c)
```

Data movement
defined from the
host perspective.

- The various forms of the map clause
 - **map(to:list)**: On entering the region, variables in the list are initialized on the device using the original values from the host (host to device copy).
 - **map(from:list)**: At the end of the target region, the values from variables in the list are copied into the original variables on the host (device to host copy). On entering the region, the initial value of the variables on the device is not initialized.
 - **map(tofrom:list)**: the effect of both a map-to and a map-from (host to device copy at start of region, device to host copy at end).
 - **map(alloc:list)**: On entering the region, data is allocated and uninitialized on the device.
 - **map(list)**: equivalent to **map(tofrom:list)**.

Vector add with dynamic memory on GPU

```
int main()
{
    float *a    = (float *) malloc(sizeof(float) * N);
    float *b    = (float *) malloc(sizeof(float) * N);
    float *c    = (float *) malloc(sizeof(float) * N);
    float *res  = (float *) malloc(sizeof(float) * N);
    int err=0;

    // fill the arrays <<<code not shown>>>

    // add two vectors
    #pragma omp target map(to: a[0:N],b[0:N]) map (tofrom: c[0:N])
    #pragma omp loop
    for (int i=0; i<N; i++){
        c[i] = a[i] + b[i];
    }

    // test results <<<code not shown>>>

    printf("vectors added with %d errors\n", err);
    return 0;
}
```

Commonly used clauses on target and loop constructs

- The basic construct* is:

#pragma omp target [*clause*[[,*clause*]]...]

#pragma omp loop [*clause*[[,*clause*]]...]

for-loops

- The most commonly used clauses are:
 - **map(to | from | tofrom list)** ← default is tofrom
 - **private(list) firstprivate(list) lastprivate(list) shared(list)**
 - behave as data environment clauses in the rest of OpenMP, but note values are only created or copied into the region, not back out “at the end”.
 - **reduction(reduction-identifier : list)**
 - behaves as in the rest of OpenMP
 - **collapse(n)**
 - Combines loops before the distribute directive splits up the iterations between teams

Going beyond simple vector addition ...

**Using OpenMP for GPU application
programming ... the heat diffusion problem**

5-point stencil: the heat program

- The heat equation models changes in temperature over time.

$$\frac{\partial u}{\partial t} - \alpha \nabla^2 u = 0$$

- We'll solve this numerically on a computer using an explicit **finite difference** discretisation.
- $u = u(t, x, y)$ is a function of space and time.
- Partial differentials are approximated using diamond difference formulae:

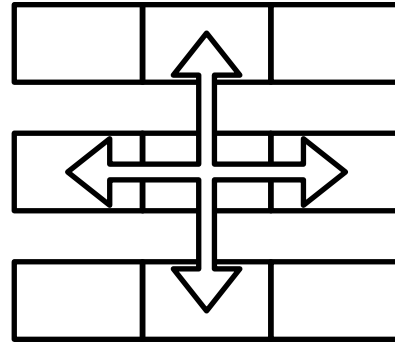
$$\frac{\partial u}{\partial t} \approx \frac{u(t + 1, x, y) - u(t, x, y)}{dt}$$

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u(t, x + 1, y) - 2u(t, x, y) + u(t, x - 1, y)}{dx^2}$$

- Forward finite difference in time, central finite difference in space.

5-point stencil: the heat program

- Given an initial value of u , and any boundary conditions, we can calculate the value of u at time $t+1$ given the value at time t .
- Each update requires values from the north, south, east and west neighbours only:



- Computation is essentially a weighted average of each cell and its neighbouring cells.
- If on a boundary, look up a boundary condition instead.

Method of Manufactured Solution

- Stencil codes are notoriously difficult to **know** if the answer is “correct”.
- Analytic solutions hard to come by:
 - It’s why you’re using a computer to solve the equation approximately after all!
- Method of Manufactured Solution (MMS) is a way to help determine if the code does the correct thing.
- An approach often used to find errors in CFD codes and check convergence properties.

Method of Manufactured Solution

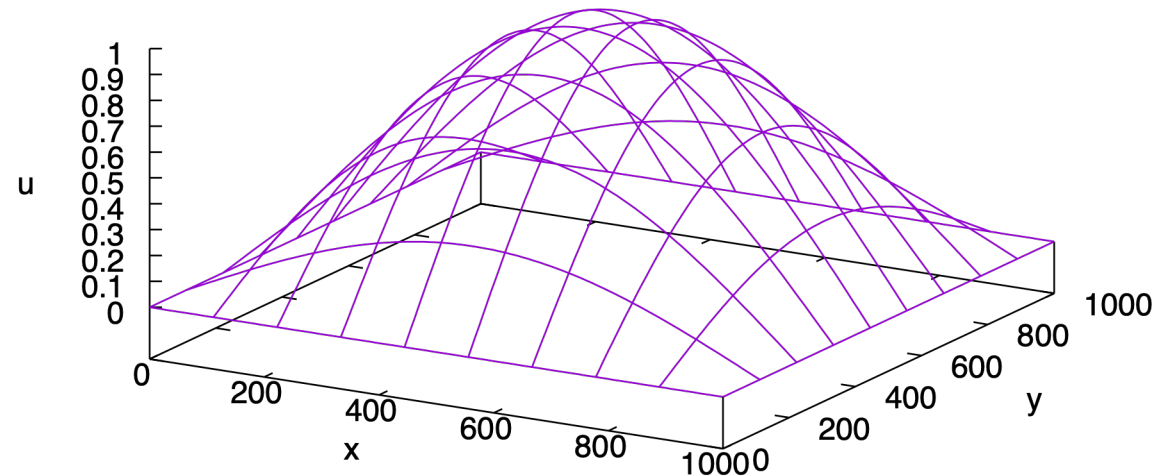
- **Choose** a function for $u(t, x, y)$, substitute into the equation and work through the algebra.
- Ideally like the equation to evaluate to zero so don't need to consider a right-hand side to the equation.
- $u(0, x, y)$ gives the initial conditions.
- Can evaluate boundary conditions, e.g. bottom boundary $u(0, 0, y)$
- Because u is **known** for all timesteps (it was chosen!), the exact solution is **known**.
- Compare the **computed** solution to the known u to compute an error.
- Any differences come from approximations in the method, or a bug in your code.

Method of Manufactured Solution

- For the problem of length l , choose u :

$$u(t, x, y) = e^{\frac{-2\alpha\pi^2 t}{l^2}} \sin \frac{\pi x}{l} \sin \frac{\pi y}{l}$$

- Boundary conditions: u is always zero on the boundaries
- Initial value of grid is then $u(0, x, y) = \sin \frac{\pi x}{l} \sin \frac{\pi y}{l}$



Heat program ...

```
// Loop over time steps
for (int t = 0; t < nsteps; ++t) {

    // solve over spatial domain for step t
    solve(n, alpha, dx, dt, u, u_tmp);

    // Pointer swap to get ready for next step
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
}
```

Note: Swapping pointers on the host before entering the target region on the next iteration works on a GPU.

When you map pointers between the host and the device, OpenMP remembers the address.

Swapped addresses on the hosts swaps addresses on the device

- Takes two optional command line arguments: <ncells> <nsteps>
 - E.g. `./heat 1000 10`
 - 1000x1000 cells, 10 timesteps (the default problem size).
- If no command line arguments are provided, it uses a default:
 - These two commands both run the default problem size of 1000x1000 cells, 10 timesteps.
 - `./heat`
 - `./heat 1000 10`
- A sensible bigger problem is 8000 x 8000 cells and 10 timesteps.

Heat diffusion problem: 5-point stencil code

```
const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;
```

```
// malloc and initialize u_tmp and u (code not shown)
```

```
for (int t = 0; t < nsteps; ++t) {
```

Loop over time steps

```
    for (int i = 0; i < n; ++i) {
```

```
        for (int j = 0; j < n; ++j) {
```

Loop over NxN spatial domain

```
            u_tmp[i+j*n] = r2 * u[i+j*n]
                        +
                        r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
                        r * ((i > 0) ? u[i-1+j*n] : 0.0) +
                        r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
                        r * ((j > 0) ? u[i+(j-1)*n] : 0.0);
```

Update the 5-point stencil. Boundary conditions on the edges of the domain are fixed at zero.

```
        }
```

```
    }
```

```
    // Pointer swap to get ready for next step
```

```
    tmp = u;
```

```
    u = u_tmp;
```

```
    u_tmp = tmp;
```

```
}
```

```
}
```

Heat diffusion problem: 5-point stencil code

```
const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;
```

```
// malloc and initialize u_tmp and u (code not shown)
```

```
for (int t = 0; t < nsteps; ++t) {
```

```
    #pragma omp parallel for collapse(2)
```

```
    for (int i = 0; i < n; ++i) {
```

```
        for (int j = 0; j < n; ++j) {
```

```
            u_tmp[i+j*n] = r2 * u[i+j*n] +
                r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
                r * ((i > 0) ? u[i-1+j*n] : 0.0) +
                r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
                r * ((j > 0) ? u[i+(j-1)*n] : 0.0);
```

```
        }
```

```
    }
```

```
    // Pointer swap to get ready for next step
```

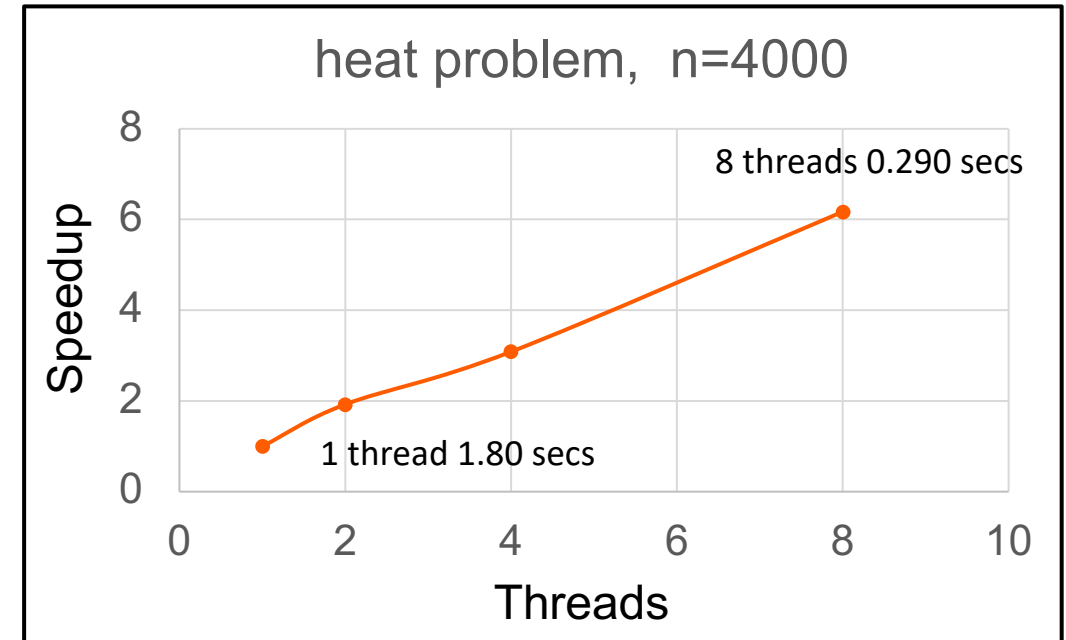
```
    tmp = u;
```

```
    u = u_tmp;
```

```
    u_tmp = tmp;
```

```
}
```

```
}
```



Intel® Xeon™ Gold 5218 @ 2.3 Ghz, 8 cores. Nvidia HPC Toolkit compiler
nvc -fast -fopenmp heat.c

Exercise: parallel stencil (heat)

- Take the provided heat stencil code (heat.c)
- Add OpenMP directives to parallelize the loops on the GPU
- Most of the runtime occurs in the solve() routine. Focus on that function. The rest of the code is there to just support the work inside solve.

- double omp_get_wtime();
- #pragma omp parallel
- #pragma omp for
- #pragma omp parallel for
- #pragma omp task
- #pragma omp taskwait
- #pragma single
- #pragma omp target
- #pragma omp loop
- Plus the clauses

- private(), firstprivate(), reduction(+:var), collapse(n)
- map(to:vp[Lower:Count]) map(from:vp[Lower:Count]) map(tofrom:vp[Lower:Count])

```
module load nvhpc/24.5
nvc -O3 -mp=gpu heat.c -o heat
sbatch job_gpu.sh
```

```
#!/bin/bash
#SBATCH -p sched_mit_psfc_gpu_r8
#SBATCH --reservation=cpsfr_2025
#SBATCH -o gpu-job.log-%j
#SBATCH --gres=gpu:1
module load nvhpc/24.5
OMP_TARGET_OFFLOAD=MANDATORY ./heat
```

job_gpu.sh

Default is tofrom: map(vp[Lower:Count])

```
salloc -p sched_mit_psfc_gpu_r8 --reservation=cpsfr_2025 --time=01:00:00
```

Heat diffusion problem: 5-point stencil code

```

const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;

// malloc and initialize u_tmp and u (code not shown)

for (int t = 0; t < nsteps; ++t) {
    #pragma omp target map(tofrom: u[0:n*n], u_tmp[0:n*n])
    #pragma omp loop
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            u_tmp[i+j*n] = r2 * u[i+j*n]
                           +
                           r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
                           r * ((i > 0) ? u[i-1+j*n] : 0.0) +
                           r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
                           r * ((j > 0) ? u[i+(j-1)*n] : 0.0);
        }
    }
    // Pointer swap to get ready for next step
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
}
}

```

When you map pointers between the host and the device, OpenMP remembers the address.

Swapped addresses on the hosts swaps
addresses on the device

GPU Solver time = 1.40 secs

This isn't much better than the
runtime for a single CPU (1.8 secs)
and worse than 8 cores on a CPU
(0.29 secs).

Why is the performance so bad?

NVIDIA T4 GPU, 16 Gbyte, Turing Arch.
Nvidia HPC Toolkit compiler
nvc -fast -mp=gpu -gpu=cc75 heat.c

Heat diffusion problem: 5-point stencil code

```
const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;

// malloc and initialize u_tmp and u (code not shown)
```

```
for (int t = 0; t < nsteps; ++t) {
    #pragma omp target map(tofrom: u[0:n*n], u_tmp[0:n*n])
    #pragma omp loop
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            u_tmp[i+j*n] = r2 * u[i+j*n]
                + r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
                r * ((i > 0) ? u[i-1+j*n] : 0.0) +
                r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
                r * ((j > 0) ? u[i+(j-1)*n] : 0.0);
        }
    }
    // Pointer swap to get ready for next step
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
}
}
```

With a runtime of 1.4 secs (worse than the CPU time) we see that Data Movement dominates performance.

At the beginning of each iteration, **copy** $(2*N^2)*\text{sizeof}(\text{TYPE})$ bytes **to** the device

We need to create a **data region** on the GPU that is distinct from the target region.

That way, we can keep the data on the device between target constructs

At the end of each iteration, **copy** $(2*N^2)*\text{sizeof}(\text{TYPE})$ bytes **from** the device

Finer control over data movement

- Recall that data is mapped to/from the device at start/end of target region
 - `#pragma omp target map(tofrom: A[0:N])`
 {
 ...
 }
- Inefficient to move data around all the time
- We need to keep data resident on the device *between* target regions.
- We need to manage the device data environment independently from the target regions that launch kernels.

Target enter/exit data constructs

- Create a data region on the target device (a device data environment) with two standalone directives:

```
#pragma omp target enter data map(...)  
#pragma omp target exit data map(...)
```

- The **target enter data** maps variables to the device data environment.
- The **target exit data** unmaps variables from the device data environment.
- Once created, subsequent **target** regions inherit the existing data environment.

Target enter/exit data example

```
void init_array(int *A, int *B, int N) {  
    for (int i = 0; i < N; ++i) { A[i] = i; B[i]=2*i;}
```

```
    #pragma omp target enter data map(to: A[0:N], B[0:N])  
}
```

```
int main(void) {  
  
    int N = 1024;  
    int *A = malloc(sizeof(int) * N);  
    int *B = malloc(sizeof(int) * N);  
    init_array(A, B, N);
```

```
    #pragma omp target
```

```
    #pragma omp loop
```

```
    for (int i = 0; i < N; ++i)  
        A[i] = A[i] * B[i];
```

```
    #pragma omp target exit data map(from: A[0:N])  
}
```

Exercise

- Modify your parallel heat code from the last exercise.
- Use the 'target data' constructs to control the device data environment.
- Think about the map clauses and how to use them to minimize data movement.
- Question ... will the pointer swap on the host still work?

- #pragma omp target
- #pragma omp target enter data map(to: list)
- #pragma omp target exit data map(from: list)
- #map(to:list) map(from:list) map(tofrom:list)
- #pragma omp target teams loop

```
module load nvhpc/24.5
nvc -O3 -mp=gpu heat.c -o heat
sbatch job_gpu.sh
```

```
#!/bin/bash
#SBATCH -p sched_mit_psfc_gpu_r8
#SBATCH --reservation=cpsfr_2025
#SBATCH -o gpu-job.log-%j
#SBATCH --gres=gpu:1
module load nvhpc/24.5
OMP_TARGET_OFFLOAD=MANDATORY ./heat
```

Heat diffusion problem: 5-point stencil code

```

const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;
// malloc and initialize u_tmp and u (code not shown)
#pragma omp target enter data map(to: u[0:n*n], u_tmp[0:n*n])

for (int t = 0; t < nsteps; ++t) {
    #pragma omp target
    #pragma omp loop
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            u_tmp[i+j*n] = r2 * u[i+j*n] +
                r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
                r * ((i > 0) ? u[i-1+j*n] : 0.0) +
                r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
                r * ((j > 0) ? u[i+(j-1)*n] : 0.0);
        }
    }
    // Pointer swap to get ready for next step
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
}
#pragma omp target exit data map(from: u[0:n*n])
}

```

Create a data region and map indicated data on entry

GPU Solver time* = **0.42** secs

This is a general principal ...
if you want performance, you
must optimize data
movement.

*includes time for target enter/exit data

Exit the data
region and map
indicated data

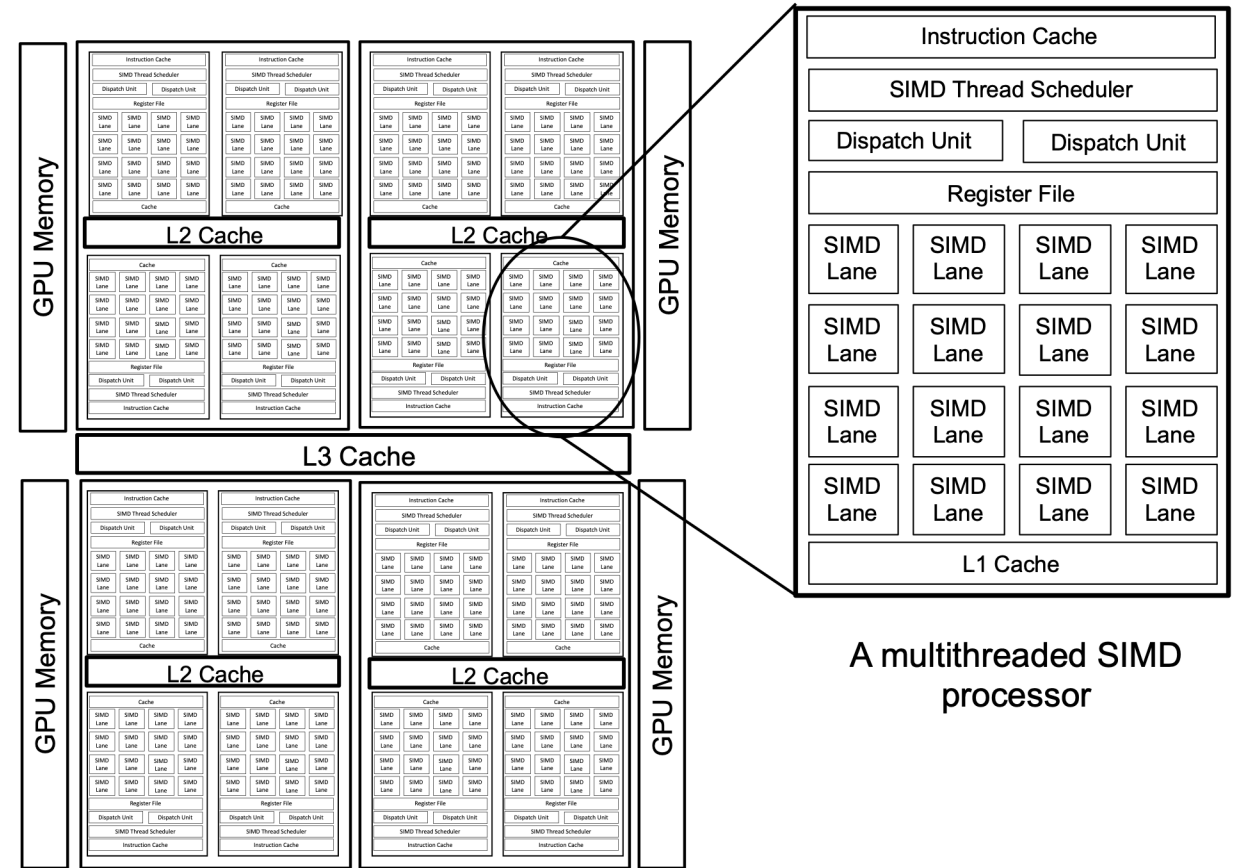
NVIDIA T4 GPU, 16 Gbyte, Turing Arch.
Nvidia HPC Toolkit compiler
nvc -fast -mp=gpu -gpu=cc75 heat.c

Getting the data movement between host memory and device memory is key.

What are the other major issues to consider when optimizing performance?

Occupancy: Keep all the GPU resources busy

- In our “GPU cartoon” we have 16 multithreaded SIMD processors each with 16 SIMD lanes For a total of $16^2=256$ processing elements.
- You want all resources busy at all times. You do that by keeping excess work for the multithreaded SIMD processors ... if they are other busy on some high latency operation, you want a new work-group is ready to be scheduled for execution.
- Occupancy having enough work-groups to keep the GPU busy. To support high occupancy, you need many more work-items than SIMD-lanes.



```
#pragma omp parallel for  
for(int i=0;i<N;i++)  
  for(int j=0;j<N;j++)  
    for(int k=0;k<N;k++)  
      *(C+(i*N+j)) += *(A+(i* N +k)) * *(B+(k* N +j));
```

**Parallelize i-loop
parallelism $O(N)$**

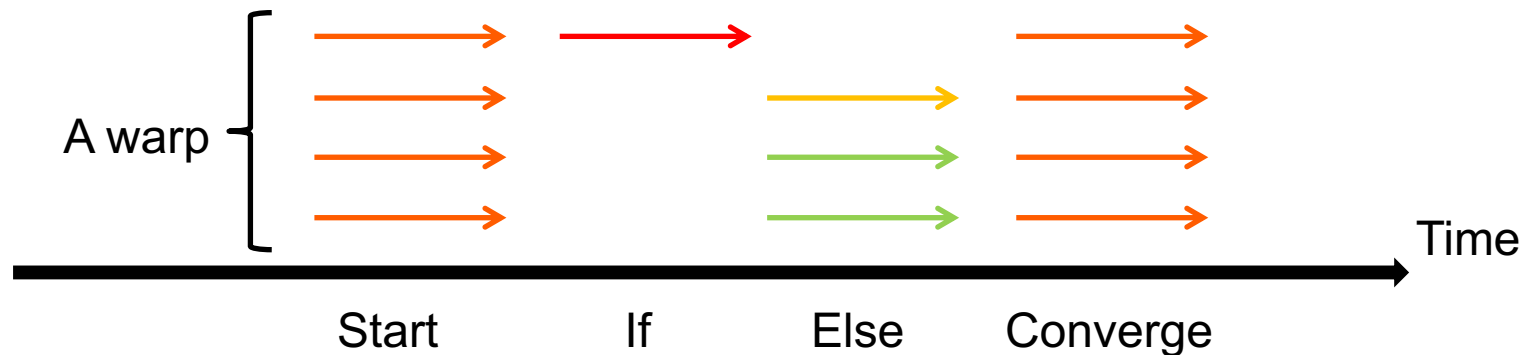


```
#pragma omp parallel for collapse(2)  
for(int i=0;i<N;i++)  
  for(int j=0;j<N;j++)  
    for(int k=0;k<N;k++)  
      *(C+(i*N+j)) += *(A+(i* N +k)) * *(B+(k* N +j));
```

**Parallelize combined i/j-loops
parallelism $O(N^2)$**

Converged Execution: Single Instruction Multiple Data

- Individual work-items of a warp start together at the same program address
- Each work-item has its own instruction address counter and register state
 - Each work-item is free to branch and execute independently
 - Supports the Single Program Multiple Data (SPMD) pattern.
- Branch behavior
 - Each branch will be executed serially
 - Work-items not following the current branch will be disabled



Converged Execution: Branching

- GPUs tend not to support speculative execution, which means that branch instructions have high latency
- This latency can be hidden by switching to alternative work-items/work-groups, but avoiding branches where possible is still a good idea to improve performance
- When different work-items executing within the same SIMD ALU array take different paths through conditional control flow, we have **divergent branches** (vs. **uniform branches**)
- Divergent branches are bad news: some work-items will stall while waiting for the others to complete
- We can use predication, selection and masking to convert conditional control flow into straight line code and significantly improve the performance of code that has lots of conditional branches

Branching

Conditional execution

```
// Only evaluate expression  
// if condition is met  
if (a > b)  
{  
    acc += (a - b*c);  
}
```

Selection and masking

```
// Always evaluate expression  
// and mask result  
temp = (a - b*c);  
mask = (a > b ? 1.f : 0.f);  
acc += (mask * temp);
```

Coalesced memory accesses

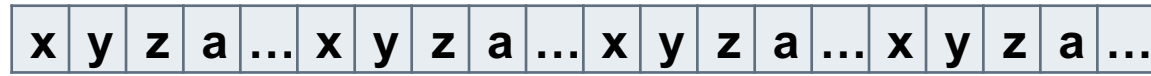
- Coalesced memory accesses are key for high performance code, especially on GPUs
- In principle, it's very simple, but frequently requires transposing or transforming data on the host before sending it to the GPU
- Sometimes this is an issue of Array of Structures vs. Structure of Arrays (AoS vs. SoA)

Memory layout is critical to performance

- Structure of Arrays vs. Array of Structures

- Array of Structures (AoS) more natural to code:

```
struct Point{ float x, y, z, a; };  
Point *Points;
```



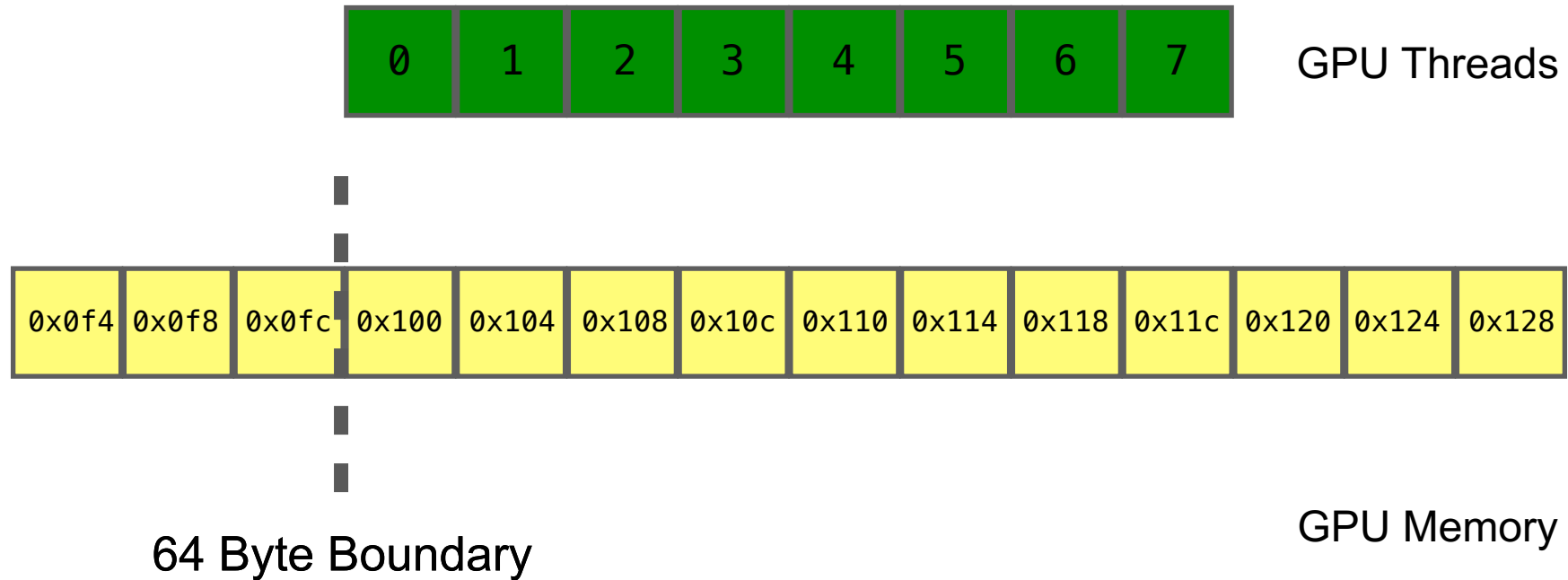
- Structure of Arrays (SoA) suits memory coalescence in vector units

```
struct { float *x, *y, *z, *a; } Points;
```



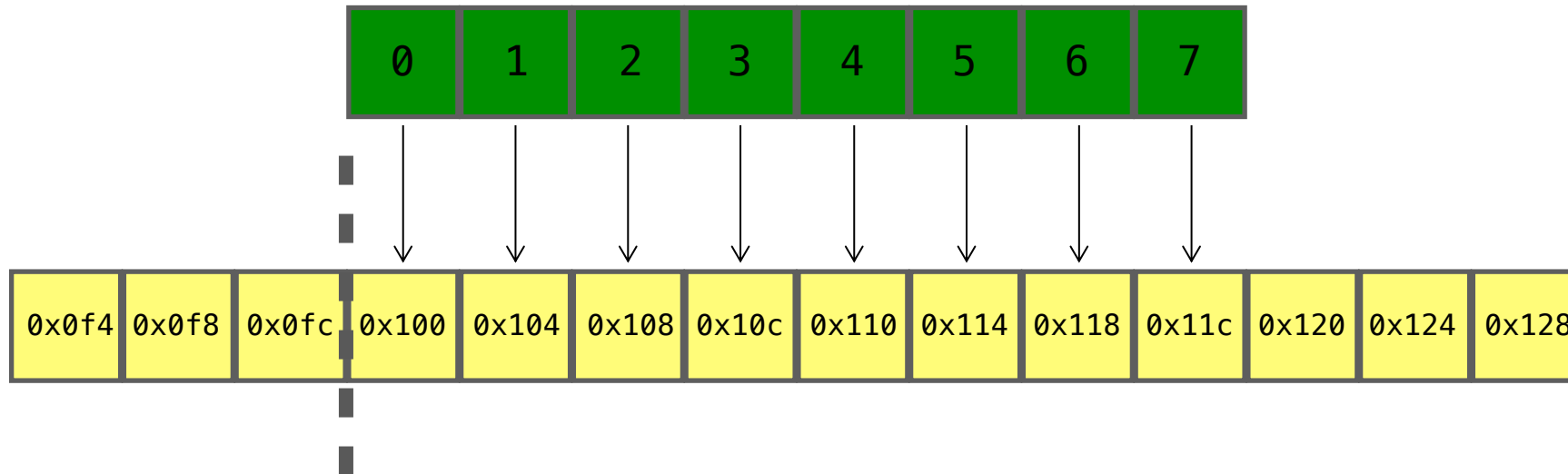
Adjacent work-items/vector-lanes like to access adjacent memory locations

Memory access patterns



Memory access patterns

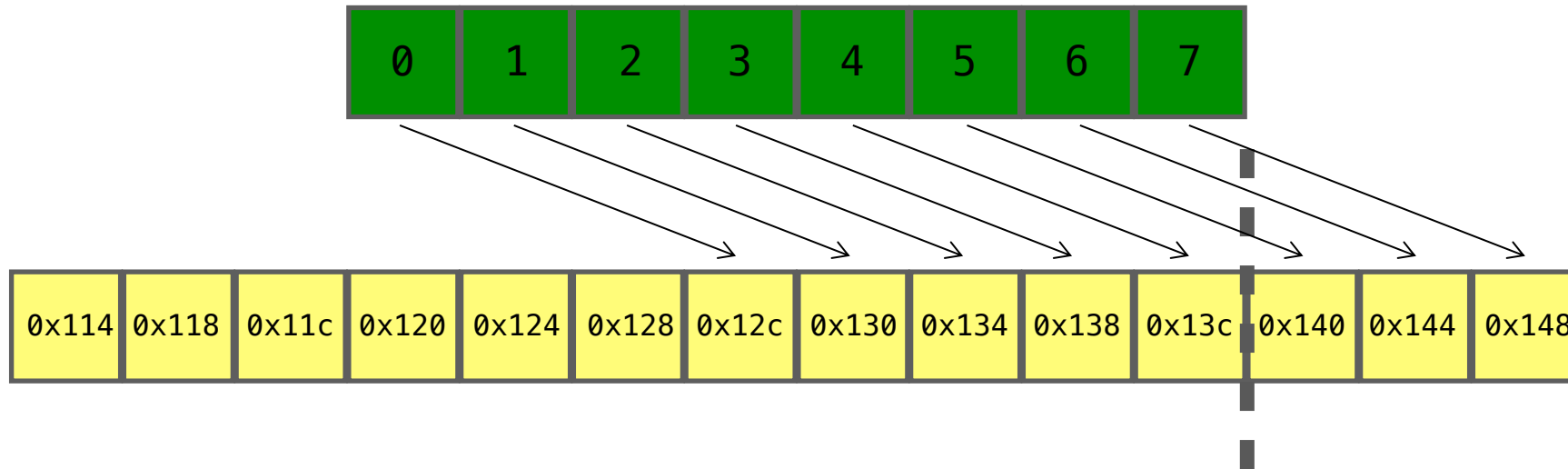
```
float val1 = memA[id];
```



64 Byte Boundary

Memory access patterns

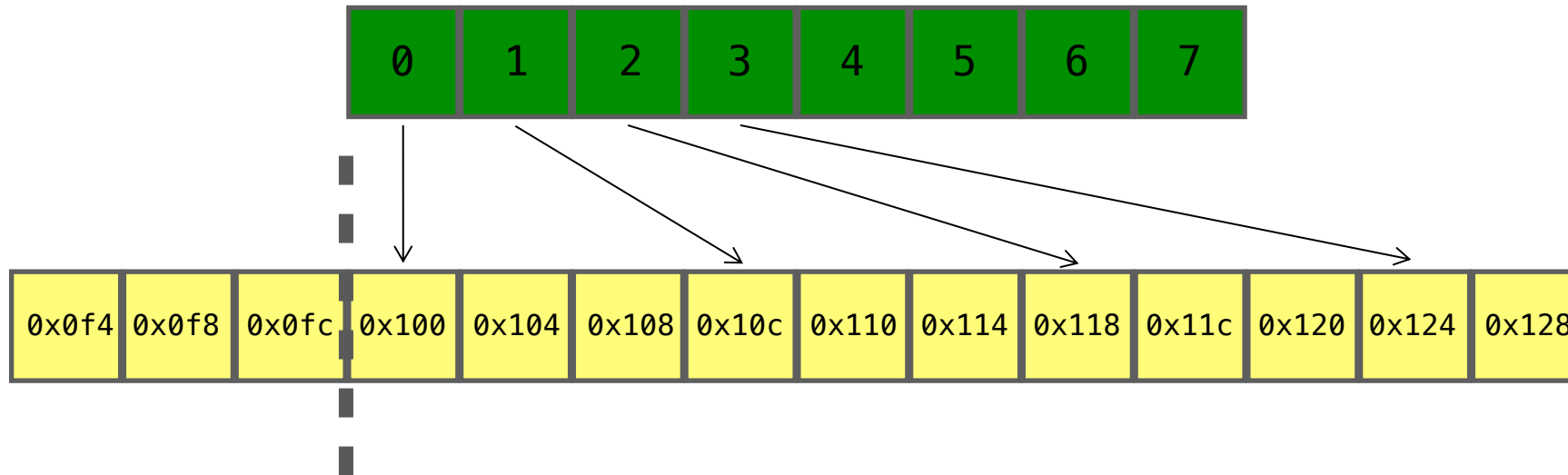
```
const int c = 3;  
float val2 = memA[id + c];
```



64 Byte Boundary

Memory access patterns

```
float val3 = memA[3*id];
```

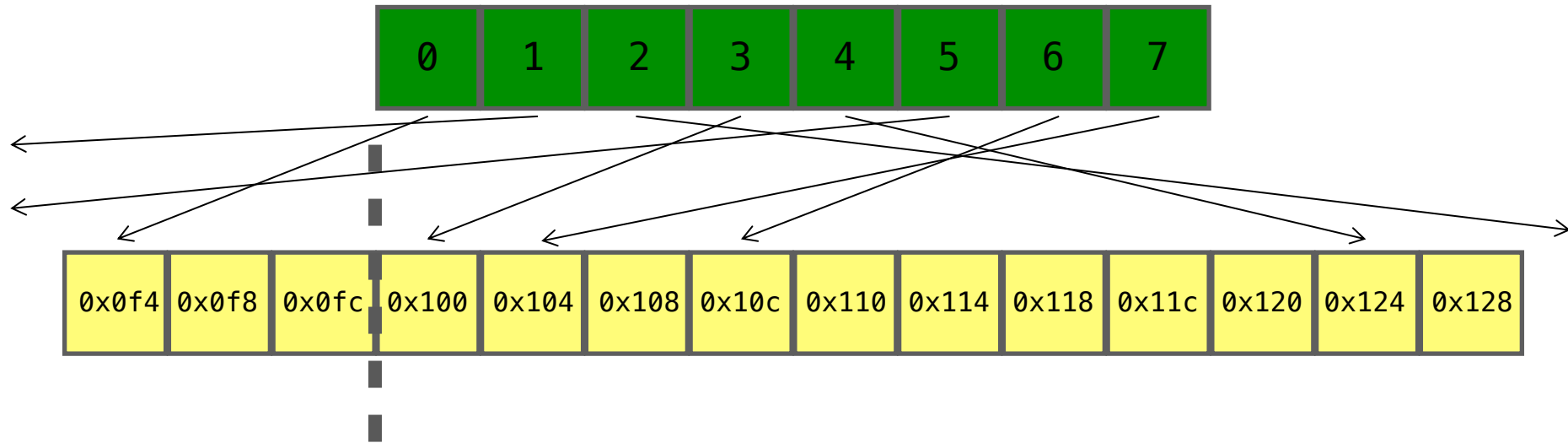


64 Byte Boundary

Strided access results in multiple
memory transactions (and
kills throughput)

Memory access patterns

```
const int loc =  
    some_strange_func(id);  
  
float val4 = memA[loc];
```



64 Byte Boundary

Coalescence

- **Coalesce** - to combine into one
- Coalesced memory accesses are key for high bandwidth
- Simply, it means, if thread i accesses memory location n then thread $i+1$ accesses memory location $n+1$
- In practice, it's not quite as strict...

```
for (int id = 0; id < size; id++)  
{  
    // ideal  
    float val1 = memA[id];  
  
    // still pretty good  
    const int c = 3;  
    float val2 = memA[id + c];  
  
    // stride size is not so good  
    float val3 = memA[c*id];  
  
    // terrible  
    const int loc =  
        some_strange_func(id);  
  
    float val4 = memA[loc];  
}
```

Exercise

- Optimize the stencil 'solve' kernel.
- Start with your code with optimized memory movement from the last exercise.
- Experiment with the optimizations we've discussed.
- Focus on the memory access pattern.
- Try different input sizes to see the effect of the optimizations.
- Keep an eye on the solve time as reported by the application.

- #pragma omp target
- #pragma omp target enter data map(to: list)
- #pragma omp target exit data map(from: list)
- #map(to:list) map(from:list) map(tofrom:list)
- #pragma omp target teams loop

```
module load nvhpc/24.5
nvc -O3 -mp=gpu heat.c -o heat
sbatch job_gpu.sh
```

```
#!/bin/bash
#SBATCH -p sched_mit_psfc_gpu_r8
#SBATCH --reservation=cpsfr_2025
#SBATCH -o gpu-job.log-%j
#SBATCH --gres=gpu:1
module load nvhpc/24.5
OMP_TARGET_OFFLOAD=MANDATORY ./heat
```

Heat diffusion problem: 5-point stencil code

Parallel CPU and GPU results, n=4000

```
const double r = alpha * dt / (dx * dx);
const double r2 = 1.0 - 4.0*r;
// malloc and initialize u_tmp and u (code not shown)
```

```
for (int t = 0; t < nsteps; ++t) {
```

```
    #pragma omp parallel for
```

```
    for (int i = 0; i < n; ++i) {
```

```
        for (int j = 0; j < n; ++j) {
```

```
            u_tmp[i+j*n] = r2 * u[i+j*n] +
                r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
                r * ((i > 0) ? u[i-1+j*n] : 0.0) +
                r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
                r * ((j > 0) ? u[i+(j-1)*n] : 0.0);
```

```
        }
```

```
    }
```

```
    // Pointer swap for next step
```

```
    tmp = u;
```

```
    u = u_tmp;
```

```
    u_tmp = tmp;
```

```
}
```

```
}
```

This is the ij loop order. Swap these loops to get the ji order.

What if I optimize the CPU code?
... just swap the i and j loops. Using collapse(2) did not help on the GPU or the CPU

CPU	Num threads	ij loop order	ji loop order
	1	1.512849	0.262260
	2	0.776229	0.132453
	4	0.400822	0.064220
	8	0.227317	0.046586
Intel® Xeon™ Gold 5218 @ 2.3 Ghz, 8 cores. Nvidia HPC Toolkit compiler nvc -fast -fopenmp heat.c			

All times in seconds

GPU	ij without timing enter and exit data	ij loop order	ji without timing enter and exit data	ji loop order
	0.056830	0.417887	0.020123	0.358905

This GPU code used the target enter data and target exit data

NVIDIA T4 GPU, 16 Gbyte, Turing Arch.
Nvidia HPC Toolkit compiler. nvc -fast -mp=gpu heat.c